ABSTRACT

Ramanujan's Magic Square is a full-stack web application developed to make the mathematical concept of magic squares interactive, educational, and engaging. A magic square is a numerical grid in which the sum of every row, column, and both main diagonals is equal to a common value called the magic constant. The project is inspired by Srinivasa Ramanujan, who was well known for constructing remarkable magic squares.

The system allows users to generate, learn, and solve magic squares of different orders. It supports squares from 3×3 to 10×10 using suitable construction algorithms: the Siamese method for odd-order squares, the complement-swap method for doubly-even squares, and the LUX/Strachey method for singly-even squares. It also explains why a 2×2 normal magic square is mathematically impossible.

Users can register, log in, play scrambled magic-square puzzles, use hints and undo options, and track their solving time and moves. The application includes daily challenges, global leaderboards, personal dashboards, achievement badges, and a Formula & Methods page that explains the mathematics behind the system. It also provides a Ramanujan-style date-based 4×4 magic-square generator.

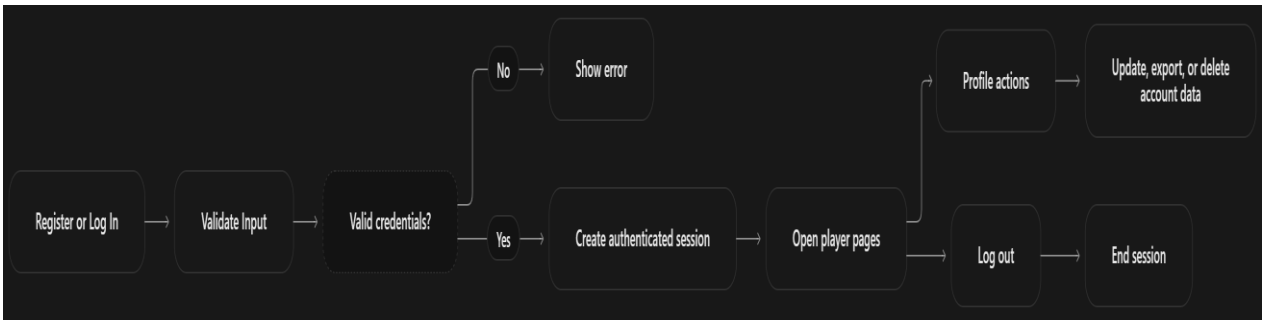
The backend is developed using Node.js and Express, while the frontend uses HTML, CSS, and JavaScript. SQLite is used for storing user accounts, game sessions, puzzle attempts, and achievements. For fairness and security, puzzle solutions and timing are validated on the server side, and passwords are securely stored using bcrypt hashing.

Overall, the project demonstrates the integration of mathematics, web development, database management, authentication, and secure game logic. It transforms magic squares from a static mathematical topic into an interactive learning platform that supports practice, competition, and conceptual understanding.

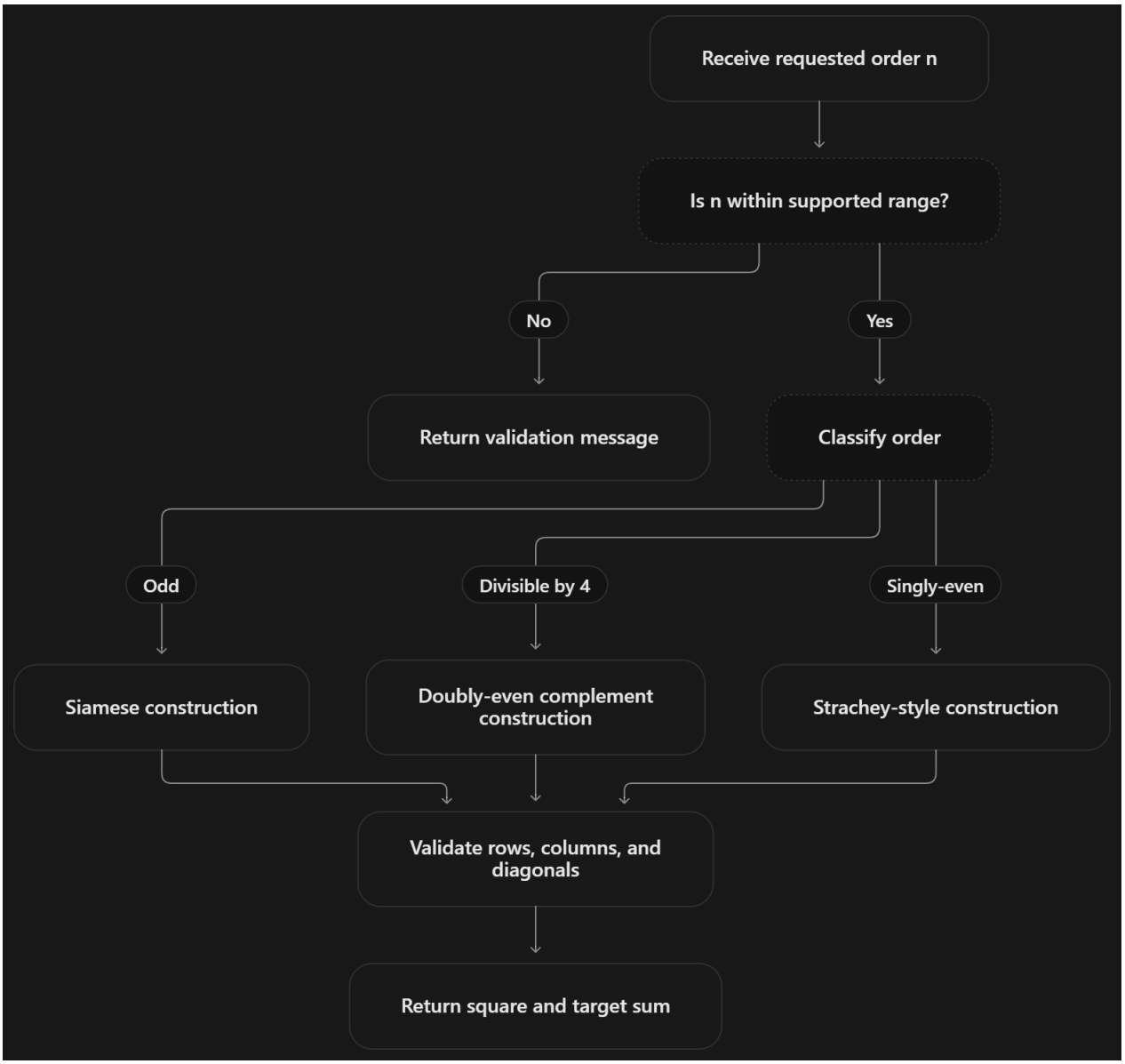
Table of Contents

CHAPTER 1: INTRODUCTION....8
1.1 Background and motivation....8
1.2 Need of Project....8
1.3 Scope of Report....9
1.4 Scope of project....9
1.5 Main Contribution....10
CHAPTER 2: PROBLEM DEFINITION AND PROJECT OBJECTIVES....11
2.1 Problem Statement....11
2.2 Project Objectives....11
CHAPTER 3: LITERATURE REVIEW....13
3.1 Historical Background of Magic Squares....13
3.2 Srinivasa Ramanujan and His Contributions....14
3.3 Mathematical Concepts Behind Magic Squares....15
3.4 Review of Existing Magic Square Systems....16
CHAPTER 4: SYSTEM ANALYSIS....17
4.1 Existing System....17
4.2 Limitations of the Existing System....18
4.3 Proposed System....19
4.4 Feasibility Study....19
CHAPTER 5: TOOLS, TECHNOLOGIES AND ALGORITHMS USED....21
5.1 Programming Languages....21
5.2 Libraries and Frameworks....22
5.3 Magic Constant Formula....23
5.4 Siamese Method....23
5.5 Doubly-Even Complement Method....24
5.6 Singly-Even LUX/Strachey Method....26
5.7 Ramanujan Date-Based Magic Square....26
CHAPTER 6: SYSTEM REQUIREMENTS....28
6.1 Hardware Requirements....28
6.2 Software Requirements....29
6.3 Functional Requirements....30

6.4 Non-Functional Requirements....31
CHAPTER 7: SYSTEM ARCHITECTURE....32
7.1 Overview of System Architecture....32
7.2 Architecture Components....32
7.3 System Workflow....34
CHAPTER 8: SYSTEM DESIGN....36
8.1 Use Case Diagram....36
8.2 Data Flow Diagram....38
8.3 Database Design....39
8.4 User Interface Design....41
CHAPTER 9: MODULE DESCRIPTION....42
9.1 User Authentication and Profile Module....42



9.2 Magic Square Generation Engine....43



9.3 Puzzle Gameplay Module....44
9.4 Hint and Undo Module....45
9.5 Daily Challenge Module....47
9.6 Leaderboard Module....48
9.7 Dashboard and Achievement Module....48
9.8 Formula and Learning Module....48
9.9 Admin Module....48
CHAPTER 10: DATABASE DESIGN AND DESCRIPTION....50
10.1 Users Table....50
10.2 Attempts table....50
10.3 Game Sessions Table....52
10.4 User Achievements Table....53
Chapter 11: System Implementation....54

11.1 Dependency and Library Setup 54
11.2 Project Folder Structure.....55
11.3 Frontend Implementation 56
11.4 Backend Implementation.....57
11.5 Database Implementation 58
11.6 Algorithm Implementation 58
11.7 Integration of Modules 60
CHAPTER 12: TESTING AND VALIDATION.....61
12.1 Test Cases and diagrams 61
CHAPTER 13: RESULTS & OBSERVATIONS.....63
13.1 Results 63
13.2 Observations.....64
13.3 Result summary.....64
CHAPTER 14: LIMITATIONS.....65
FUTURE ENHANCEMENTS.....66
CONCLUSION 67
CHAPTER 15: DASHBOARD SCREENSHOTS.....68
15.1 Home Page and Account Entry 69
15.2 User Registration and Login.....70
15.3 Player Dashboard.....71
15.4 Profile Dashboard.....72
15.5 Magic Square Gameplay 73
15.6 Daily Challenge and Leaderboard.....74
15.7 Formula and Learning Section 75
15.8 Admin Dashboard.....76
REFERENCES.....78
BIBLIOGRAPHY 80
Screenshot materials to supply 83

LIST OF FIGURES

Figure 7.1: High-level system architecture. The diagram shows how the user's browser communicates with the Flask application, database, and optional AI service.....32

Figure 7.2: Typical gameplay workflow. The flow shows the sequence from starting a puzzle to validation and progress recording. 34

Figure 8.1: Main use cases. The diagram summarizes the actions available to users and administrators. 36

Figure 8.2: Context-level data flow diagram. The diagram identifies the main information exchanged between users, the application, and stored data.....38

Figure 8.3: Simplified database relationship diagram. A single user may be associated with many gameplay and chatbot records. 39

Figure 9.2: Authentication and profile flow. This diagram outlines the main account workflow.42

Figure 9.3: Square generation flow. The order determines which construction method the engine applies.....43

Figure 9.5: Daily Challenge flow. 47

Figure 9. User-supplied handmade model of the 139 square 52

Figure 15.1: Home page and account entry screen.....69

Figure 15.2: User registration and login screen.....70

Figure 15.3: Player dashboard showing progress and achievements. 71

Figure 15.4: Profile dashboard and account-management actions. 72

Figure 15.5: Magic-square gameplay interface. 73

Figure 15.6: Daily Challenge and leaderboard screens. 74

Figure 15.7: Formula and learning section. 75

Figure 15.8: Administrator dashboard and review interface. 76

LIST OF TABLES

Table 1: Lo Shu normal 3×3 magic square 23
Table 2: Sequential arrangement used to construct a normal 4×4 square 24
Table 3: Completed normal 4×4 magic square 25
Table 4: Ramanujan's date-based 4×4 magic square 26
Table 5: Users table schema 50
Table 6: Attempts table schema 50
Table 7: Game sessions table schema 52
Table 8: User achievements table schema 53
Table 9: Functional test cases and expected outcomes 61
Table 10: Project capability summary 64
Table 11: Ramanujan date-square reference grid 81
Table 12: Magic constants for supported normal-square orders 81
Table 13: Application environment variables 82

CHAPTER 1: INTRODUCTION

1.1 Background and motivation

Magic squares are mathematical grids in which numbers are arranged so that every row, column, and both main diagonals have the same total value. This common total is called the magic constant. Magic squares have been studied for centuries in mathematics, puzzles, number theory, and recreational learning. They are useful for introducing concepts such as patterns, logical reasoning, algorithms, validation, and problem solving. Srinivasa Ramanujan, one of India's greatest mathematicians, was especially known for his ability to create magic squares, including squares related to dates. His work inspired the theme of this project.

Traditionally, magic squares are studied from textbooks or solved manually on paper. Although this helps learners understand the final arrangement, it does not always explain how a square is constructed or allow them to practise interactively. Different square sizes require different generation methods. For example, odd-order magic squares can be generated using the Siamese method, while even-order squares require other methods. Therefore, creating a digital system for magic squares involves both mathematical logic and software development.

Ramanujan's Magic Square is a web application that combines these two areas. It generates magic squares, scrambles them into playable puzzles, checks whether the user's solution is correct, and stores performance details. The application supports square orders from 3×3 to 10×10. It also explains that a normal 2×2 magic square is mathematically impossible. The project uses Node.js and Express for the backend, SQLite for data storage, and HTML, CSS, and JavaScript for the user interface. It therefore provides a practical example of how mathematical concepts can be transformed into an interactive full-stack application

1.2 Need of Project

The need for this project comes from the difficulty students may face when learning mathematical patterns only through static examples. A magic square shown in a book gives the final answer, but it may not show the construction process, the rules behind it, or the method used to verify it. Learners may understand that all rows and columns should have the same sum, but they may not get an opportunity to apply this knowledge in an engaging way. This project makes the learning process more active by allowing users to generate, solve, and study magic squares through a browser.

The project is also needed because it connects mathematics with programming concepts. It demonstrates how algorithms can generate correct solutions automatically. For example, the system uses different generation methods for odd-order, doubly-even, and singly-even magic squares. This shows that one mathematical problem may require different algorithmic approaches depending on the input size. The application also calculates the magic constant using the formula $M(n) = n(n^2 + 1)/2$ and validates the generated result.

Another important need is performance tracking. In manual puzzles, it is difficult to keep records of time taken, moves made, or improvement over multiple attempts. This system stores verified puzzle attempts and shows users their best times and solving history. Daily challenges, achievements, streaks, and leaderboards make the experience more motivating. The project also uses server-side validation, so users cannot simply submit a false time or an incorrect grid. Therefore, the system is useful not only as a learning tool but also as a secure and competitive puzzle platform.

1.3 Scope of Report

This report covers the design and implementation of the Ramanujan's Magic Square web application. It explains the mathematical concept of magic squares and how the application converts this concept into an interactive puzzle system. The report includes the purpose of the project, problem statement, objectives, required hardware and software, system design, database tables, modules, and expected results. It also describes the technologies used to build the frontend, backend, and database.

The project scope includes generating valid magic squares for orders 3×3 to 10×10. It includes gameplay where a user receives a scrambled square and must restore the correct arrangement by swapping tiles. The system records the time taken, moves made, hints used, and whether the attempt was part of a daily challenge. It also includes user registration, login, password management, account settings, personal dashboards, achievements, leaderboards, and administration functions.

The report further covers the database structure used to store users, game sessions, attempts, and achievements. It explains how the server validates puzzle completion to maintain fair scores. The project also includes an educational Formula and Methods page, where users can learn the magic constant formula and the different algorithms used for generating magic squares. A Ramanujan-style date-based 4×4 magic square generator is also included.

However, the report does not focus on advanced multiplayer gaming, payment systems, social-media login, native mobile applications, or large-scale distributed deployment. The current system is designed as a single web application using SQLite and in-memory sessions. Thus, the report mainly focuses on educational puzzle functionality, secure user management, mathematical correctness, and full-stack web development.

1.4 Scope of project

The project focuses on designing and developing a web-based learning and puzzle platform centered on Ramanujan's Magic Square and the broader study of magic squares. It presents the mathematical idea of arranging numbers in a square so that selected rows, columns, and diagonals have equal sums, and provides explanations of the magic constant and construction methods for different square orders. Users can explore the special 4×4 square associated with Ramanujan, including its date-based arrangement and the ways its entries form equal-sum groups. The application supports interactive puzzle solving through Swap, Fill, and Manual modes, with features such as hints, undo, timers, move tracking, and validation of completed boards. It includes Daily Challenges, progress tracking,

achievements, attempt history, and leaderboards to encourage continued practice and let users review their performance. Account features allow users to register, log in, manage profile details, change passwords, export their data, and delete their accounts. A bilingual chatbot provides help about the magic-square game and the website in English or Kannada; when configured, it can use an AI service together with relevant project guide content, while a curated response system remains available as a fallback. The system uses a web frontend, a Python Flask backend, and SQLite for persistent user and gameplay data. The project covers the generation, presentation, validation, and educational use of magic squares within the website. It does not provide live multiplayer matchmaking, voice chat, screenshot analysis, or support for unrelated games. Its intended scope is to make magic-square concepts accessible through a practical, interactive application that can be used for learning, practice, and performance tracking.

1.5 Main Contribution

The main contribution of this project is the conversion of a mathematical topic into an interactive educational web application. Instead of presenting magic squares only as fixed number patterns, the system allows users to generate them, observe their logic, solve them as puzzles, and measure their performance. This makes the topic more engaging for students and users who enjoy logical games. It also helps users understand that mathematics can be applied through software development.

A major contribution is the implementation of multiple magic-square generation algorithms. The project uses the Siamese method for odd-order squares such as 3×3, 5×5, 7×7, and 9×9. It uses the complement-swap method for doubly-even orders such as 4×4 and 8×8. For singly-even orders such as 6×6 and 10×10, it uses the LUX/Strachey approach. This allows the application to support a wider range of square sizes while maintaining mathematical correctness.

Another contribution is secure gameplay validation. When a puzzle is started, the server stores the expected solution and the starting time. When the user submits the completed grid, the server checks it against the stored solution and calculates the final time itself. This prevents users from submitting incorrect solutions or fabricated scores. Passwords are securely hashed, and account recovery uses time-limited tokens.

The project also contributes useful user features such as daily challenges, global leaderboards, achievements, hints, undo options, personal solve history, and a dashboard. These features make the application more than a simple generator. Overall, Ramanujan’s Magic Square demonstrates the combination of mathematics, algorithms, database management, web development, security, and user-friendly design in one complete project.

CHAPTER 2: PROBLEM DEFINITION AND PROJECT OBJECTIVES

2.1 Problem Statement

Traditional explanations of magic squares often present formulas and solved examples without giving learners a practical way to explore how the numbers work together. As a result, students may memorize the magic constant without understanding why every row, column, and main diagonal reaches the same total.

Constructing squares of different orders can also be challenging. Learners must place each number correctly, avoid duplicates, and check several sums at once. When an arrangement is incorrect, it may not be clear which move caused the problem or how to improve it.

The project addresses these difficulties by providing a web-based platform for learning and practising Ramanujan's Magic Square and related magic-square concepts. It combines explanations of the mathematics with interactive puzzle modes, generated boards, and validation so users can learn by trying, checking, and correcting their work.

The application also aims to make practice more engaging through hints, undo, timers, Daily Challenges, achievements, leaderboards, and personal progress records. User accounts allow progress and game history to be saved and accessed again later.

Therefore, the main problem is to develop an accessible and dependable application that helps users understand magic-square construction, practise solving puzzles of different sizes, and review their performance. The system must present mathematical information clearly, provide useful feedback during gameplay, and store user and progress data reliably.

2.2 Project Objectives

The main objective of this project is to develop an interactive web-based platform that helps users learn, construct, and solve Ramanujan's Magic Square and related magic-square puzzles. The specific objectives are as follows:

- To explain the basic rules, mathematical properties, and magic constant of magic squares in a clear and accessible way.
- To present Ramanujan's 4×4 date-based magic square and demonstrate how its rows, columns, diagonals, and selected groups of numbers produce equal sums.
- To generate valid magic squares of different supported orders using suitable construction methods.
- To provide Swap, Fill, and Manual play modes so users can practise solving puzzles in different ways.
- To validate player entries and provide feedback, hints, and undo controls during gameplay.

- To offer Daily Challenges, timers, attempt history, achievements, and leaderboards to encourage regular practice and track performance.
- To provide user accounts that preserve profile information and game progress between visits.
- To offer a chatbot that answers questions about magic squares and website features in English or Kannada, with AI-powered replies available when configured.
- To store user and gameplay data reliably using a database and provide account-data export and management features.

CHAPTER 3: LITERATURE REVIEW

3.1 Historical Background of Magic Squares

A magic square is a square arrangement of numbers in which the entries in each row, each column, and the two principal diagonals have the same total. The history of magic squares stretches across several cultures, where they appeared in mathematical study, symbolic traditions, and recreational puzzles. Their precise origins are difficult to establish because early accounts often mix mathematical descriptions with legend.

One of the best-known early examples is the *Lo Shu* square from ancient China. It is a 3×3 arrangement of the numbers 1 through 9 in which every row, column, and principal diagonal sums to 15. Chinese tradition connects the pattern with a turtle emerging from the Lo River, although the story is generally treated as legend rather than verified historical evidence. The square's symmetry and numerical structure made it an influential example in later discussions of number patterns.

Magic squares were also studied in India and the Islamic world, and knowledge of mathematical techniques travelled between regions through scholarship and trade. In medieval Islamic mathematics, scholars investigated methods for constructing squares of different orders. Later, European mathematicians explored their properties and construction systematically. In 1514, the German artist Albrecht Dürer included a 4×4 magic square in his engraving *Melencolia I*. Its bottom row contains the numbers 15 and 14 in the middle, which identify the year of the engraving, making the square notable both mathematically and historically.

Over time, magic squares became a subject of recreational mathematics and formal study. Researchers examined how to construct squares, classify them, and determine whether arrangements were equivalent under rotations or reflections. The standard normal magic square of order n uses each integer from 1 to n^2 exactly once. Its common row, column, and principal-diagonal sum is called the magic constant and is given by $M = n(n^2 + 1)/2$.

Srinivasa Ramanujan is associated with a distinctive 4×4 square arranged using the digits of his date of birth. Its rows, columns, and principal diagonals sum to 139. The square is an example of a date-based construction and differs from a normal square that uses the consecutive numbers 1 through 16. This distinction illustrates the variety within the broader subject: some magic squares follow the normal-number rule, while others use different entries or are designed to highlight additional numerical relationships.

3.2 Srinivasa Ramanujan and His Contributions

Srinivasa Ramanujan (1887–1920) was an Indian mathematician whose original work made lasting contributions to number theory, infinite series, continued fractions, and mathematical analysis. Born in Erode and raised in Kumbakonam, he developed a strong interest in mathematics from a young age. Much of his knowledge came from independent study, and he recorded many results in notebooks. In 1913, he began corresponding with the British mathematician G. H. Hardy, who recognised the originality of his work and helped him travel to Cambridge. Ramanujan collaborated with Hardy and other mathematicians and was elected a Fellow of the Royal Society in 1918. He died in 1920, aged 32, leaving notebooks that continue to attract mathematical study.

Ramanujan is popularly associated with a 4×4 magic square constructed from the digits of his birth date, 22 December 1887. Its first row is **22, 12, 18, 87**, which records the date as day, month, century, and year. In the complete arrangement, each row, column, and main diagonal sums to **139**. The square also has other equal-sum groups, including the four corner entries and the central 2×2 block. This example shows how a carefully arranged set of numbers can reveal patterns beyond the standard row and column rules. It is a memorable mathematical construction associated with Ramanujan, though it should be distinguished from his broader research contributions in number theory and analysis.

Ramanujan's work includes important formulas and results concerning the partition function, which counts the ways a positive integer can be written as a sum of positive integers; continued fractions; and infinite series related to π. His collaboration with Hardy produced the Hardy–Ramanujan asymptotic formula for the partition function. The number **1729** is also linked to a well-known anecdote about Ramanujan and Hardy: it is the smallest positive integer expressible as the sum of two positive cubes in two different ways, $\mathbf{1^3 + 12^3 = 9^3 + 10^3}$. This number is not an entry in Ramanujan's 4×4 magic square; the two are connected through stories about his mathematical insight.

In this project, Ramanujan's name provides the historical context for studying the 4×4 date square and the wider topic of magic squares. The application uses that example to help users explore equal sums, number patterns, and puzzle-solving methods through interactive play. It also introduces general construction and validation concepts, showing how mathematical ideas can be presented as practical activities for learners.

3.3 Mathematical Concepts Behind Magic Squares

A magic square is a square arrangement of numbers in which the entries in every row, every column, and the two main diagonals have the same total. This common total is called the magic constant. A square of order n has n rows and n columns. In a normal magic square, the numbers from 1 to n^2 are used exactly once.

The magic constant can be derived by considering the sum of all entries. The numbers from 1 to n^2 have total

$$1 + 2 + \cdots + n^2 = \frac{n^2(n^2+1)}{2}.$$

The entries are divided among n rows, each with the same sum M. Therefore,

$$nM = \frac{n^2(n^2+1)}{2}, \quad M = \frac{n(n^2+1)}{2}.$$

For example, in a normal 4×4 magic square, the entries are 1 through 16, so the magic constant is $4(16+1)/2 = 34$. Each row, column, and main diagonal must add to 34. In a normal 3×3 square, the corresponding constant is $3(9+1)/2 = 15$. This formula gives the required sum, but it does not by itself construct a valid square.

Normal magic squares also have a useful complementary-number property. The smallest and largest entries, 1 and 16, sum to 17; the next pair, 2 and 15, also sum to 17. In general, the complementary pairs in an order-n normal square sum to $n^2 + 1$. Construction methods place these values so that the required lines achieve the magic constant.

Different orders require different construction techniques. Odd-order squares can be built with the Siamese method. Doubly-even orders, divisible by four, can be constructed using complementary values. Singly-even orders, which are even but not divisible by four, need a different arrangement, such as Strachey-style methods. Whatever method is used, a completed square can be verified by checking that it contains each required number once and that all specified lines equal the calculated magic constant.

Ramanujan's famous date-based 4×4 square is a special example rather than a normal square using 1 through 16. Its entries are based on his birth date, and its rows, columns, and main diagonals sum to 139. The normal-square complementary-pair rule of 17 does not apply to its shifted date values. Distinguishing these two kinds of squares helps explain both the general theory and the special properties of Ramanujan's arrangement.

3.4 Review of Existing Magic Square Systems

Magic-square systems are available in several forms, including printed puzzles, educational websites, mobile applications, and mathematical software. Printed puzzles are simple to use and encourage manual problem-solving, but they cannot automatically check a solution or provide immediate guidance. Basic online generators can create completed squares for different orders, yet many focus mainly on displaying results rather than teaching how the numbers are arranged or why their sums are equal.

Some educational applications provide interactive grids, hints, and answer checking. These features help learners practise, but the range of supported construction methods, puzzle modes, and progress-tracking tools can vary. General mathematical software can generate and analyse squares in detail, although it may require users to have programming or advanced mathematical knowledge.

The Ramanujan's Magic Square project combines learning material with interactive practice. It presents construction methods and mathematical properties alongside Swap, Fill, and Manual modes, validation, hints, undo, and progress features. Daily Challenges, achievements, and leaderboards add structured practice, while the chatbot provides help with the game and website. This combination aims to make the subject approachable for learners while offering practical tools to explore and solve magic squares.

CHAPTER 4: SYSTEM ANALYSIS

4.1 Existing System

Traditional magic-square learning mainly relies on textbooks, classroom explanations, and manually solved examples. These methods introduce the rules and formulas, but learners may have limited opportunities to practise constructing squares or receive immediate feedback when an arrangement is incorrect.

The existing MagicSquare website provides a browser-based platform for learning and playing. Users can register, log in, and solve puzzles using Swap, Fill, and Manual modes. The system generates boards, checks solutions, and offers features such as hints, undo, timers, and move tracking. It also includes Daily Challenges, a leaderboard, achievements, progress history, and a section explaining magic-square formulas and construction methods.

User and gameplay information is stored in an SQLite database, allowing the application to retain account details and progress between visits when the database is kept on persistent storage. A chatbot helps answer questions about the game and website in English or Kannada. When an OpenAI API key is configured, it can provide AI-generated responses grounded in the project's guide; otherwise, it uses curated responses.

This system combines mathematical learning with interactive practice, helping users explore magic squares and monitor their progress through a single web application.

4.2 Limitations of the Existing System

Traditional methods of learning and solving magic squares rely mainly on textbooks, printed examples, or manual calculations. These approaches explain the theory, but often provide limited opportunities for interactive practice and immediate feedback. Common limitations include:

- Learners may find it difficult to understand how rows, columns, diagonals, and the magic constant relate to one another from static examples alone.
- Constructing and checking larger magic squares manually can be time-consuming and prone to calculation errors.
- Printed examples do not let users rearrange numbers, test moves, or receive immediate validation.
- Learners may have little guidance when they make an incorrect move or become stuck while solving a puzzle.
- Traditional practice does not usually record solve times, moves, attempts, or progress across sessions.
- Daily challenges, achievement tracking, and leaderboards are generally unavailable in books or offline activities.
- Explanations of Ramanujan's date-based square may not clearly distinguish its special properties from those of normal magic squares.
- Learning resources, puzzle practice, and progress records are often separated, requiring learners to use multiple sources.
- Manual practice may offer limited accessibility for users who want to learn at their own pace or revisit explanations while solving.

These limitations motivate the development of an interactive platform that combines explanations, puzzle solving, validation, and progress tracking in one application.

.

4.3 Proposed System

The proposed system is a web-based platform for learning and playing Ramanujan's Magic Square. It brings mathematical explanations, interactive puzzles, and player progress tools together in one application. Users can learn the rules and construction methods, view Ramanujan's 4×4 date-based square, and practise solving boards of different sizes.

The system provides Swap, Fill, and Manual modes. It checks completed boards, gives feedback, and offers hints and undo controls to support practice. Daily Challenges, attempt history, achievements, and leaderboards help users track their activity and stay engaged.

Users can create accounts so their profile and gameplay progress are saved in the database and available when they return. A chatbot helps with questions about magic squares and website features in English or Kannada. When an OpenAI API key is configured, it can provide AI-generated answers grounded in the project's guide; otherwise, it uses curated responses.

The application uses a browser-based interface, a Python Flask backend, and SQLite for data storage. This design provides a practical way to combine learning, puzzle solving, and progress tracking in a single system.

4.4 Feasibility Study

A feasibility study evaluates whether the proposed Ramanujan's Magic Square project can be developed and operated with the available technology, resources, and user skills. The study considers its technical, economic, and operational feasibility.

4.4.1: Technical Feasibility

The project is technically feasible using commonly available web technologies. The frontend uses HTML, CSS, and JavaScript to display the game, learning pages, chatbot, and dashboards. The backend is developed with Python and Flask, which handles user authentication, game logic, data processing, and communication between the frontend and database. SQLite stores account information, gameplay attempts, achievements, and other application data. Magic-square generation and validation use mathematical algorithms that can be implemented and run on an ordinary web server. The optional AI chatbot requires a server-side OpenAI API key and an internet connection; curated responses remain available when AI is not configured. Users can access the application through a modern web browser on a computer or mobile device.

4.4.2: Economic Feasibility

The project can be developed with open-source technologies, including Python, Flask, SQLite, HTML, CSS, and JavaScript, which do not require paid software licenses. Development and local testing can be carried out using an ordinary computer and freely available tools. Hosting costs depend on the deployment provider and whether persistent storage is required. The project's configured Render deployment uses a paid web service and persistent disk to retain its SQLite database across updates. Optional AI responses may also involve API usage charges, depending on the provider's

current pricing and usage. The system can still operate with curated chatbot responses if the AI service is not enabled.

4.4.3: Operational Feasibility

The application is designed for straightforward use through familiar browser controls and clear navigation. Users can register or log in, select a game mode, solve a square, request hints, review results, and track their progress through the dashboard. Explanations and chatbot assistance help users understand the rules and features while using the system. Administrators can review relevant user and chatbot information through the admin features. Routine operation requires a running web server and access to the database; for a hosted deployment, persistent storage must remain attached so saved account and gameplay data are retained.

CHAPTER 5: TOOLS, TECHNOLOGIES AND ALGORITHMS USED

5.1 Programming Languages

The Ramanujan's Magic Square project uses **HTML, CSS, JavaScript, and Python** to build an interactive web application. Each language has a distinct role in presenting the website, handling user interactions, and implementing the game and server logic.

HTML (HyperText Markup Language) structures the application's web pages. It defines elements such as navigation menus, forms, puzzle boards, buttons, dashboard sections, and chatbot controls.

CSS (Cascading Style Sheets) controls the visual appearance and layout of the application. It is used to style the pages, puzzle grids, forms, menus, and dashboard components, and to adapt the interface for different screen sizes.

JavaScript runs in the browser and provides interactive behaviour. It manages puzzle controls, user actions, timers, hints, undo operations, board updates, and communication with the backend through web requests.

Python implements the server-side application using the Flask framework. It handles requests from the frontend, applies game and validation logic, manages user authentication, and reads or updates persistent application data.

Together, these languages support the complete application: HTML and CSS present the interface, JavaScript handles browser-side interactions, and Python processes application requests and game-related operations on the server.

5.2 Libraries and Frameworks

The project uses Flask as its backend web framework. Flask handles HTTP requests, application routes, user sessions, and communication between the browser and server. Its lightweight structure suits the project because it provides the required web features without adding unnecessary framework complexity.

The frontend is built with HTML, CSS, and JavaScript. HTML defines page content and forms, CSS controls the layout and responsive appearance, and JavaScript supports interactive gameplay, board updates, timers, and communication with backend API routes.

SQLite is used as the relational database. It stores user accounts, game attempts, sessions, achievements, and other application records in a local database file. Python's built-in SQLite support allows the Flask application to access and update this data without requiring a separate database server for local development.

The project also uses Python libraries for secure password handling and application functionality. The bcrypt library hashes passwords before storage, while Flask's session support maintains authenticated user sessions. The application uses the openai integration when an API key is configured to provide AI-assisted chatbot replies; without it, the chatbot can use its curated response system. These frameworks and libraries work together to support the website's interface, game logic, account management, data storage, and optional AI assistance.

5.3 Magic Constant Formula

The magic constant, also called the magic sum, is the common total required for every row, every column, and the two main diagonals of a normal magic square. A normal square of order n contains each integer from 1 to n^2 exactly once.

The sum of all numbers from 1 to n^2 is:

$$1 + 2 + \cdots + n^2 = \frac{n^2(n^2+1)}{2}$$

Since the square has n rows and each row must have the same total, the magic constant M is the total sum divided by n:

$$M = \frac{n^2(n^2+1)}{2n} = \frac{n(n^2+1)}{2}$$

For example, a 3×3 normal magic square has $n = 3$, so $M = 3(3^2+1)/2 = 15$. For a 4×4 normal magic square, $M = 4(4^2+1)/2 = 34$. The project uses this formula to calculate the target sum for supported board sizes and to validate completed rows, columns, and main diagonals.

This formula applies to normal magic squares, including odd, doubly-even, and singly-even orders. Although it gives $M = 5$ for $n = 2$, that value alone does not guarantee a valid 2×2 normal magic square; such a square cannot satisfy all the required conditions using the numbers 1 through 4 exactly once.

5.4 Siamese Method

The Siamese method is a systematic technique for constructing a normal magic square of odd order, such as 3×3, 5×5, or 7×7. A normal magic square of order n uses every integer from 1 to n^2 exactly once. In the completed square, each row, column, and main diagonal has the same sum, called the magic constant:

$$M = \frac{n(n^2+1)}{2}$$

To begin, place the number 1 in the middle cell of the top row. For each following number, move one cell up and one cell to the right. If the move goes beyond the top or right edge, continue from the opposite edge of the square. This wrapping keeps the move within the grid.

If the destination cell is already occupied, do not use it. Instead, move one cell down from the position containing the previous number and place the next number there. Continue this process until all numbers from 1 to n^2 have been placed.

For a 3×3 square, the method produces the following arrangement:

Table 1: Lo Shu normal 3×3 magic square

8 1 6

8 1 6

3 5 7

4 9 2

Each row, column, and main diagonal sums to 15, which agrees with the formula for n = 3. The method is suitable for odd-order squares, but it does not directly construct even-order squares; those require different construction methods.

1:54 PM

5.5 Doubly-Even Complement Method

A magic square has a doubly-even order when its order n is divisible by 4, such as 4×4, 8×8, or 12×12. One way to construct a normal doubly-even magic square is the complement method. It places the numbers from 1 to n^2 in order, then replaces selected entries with their complements so every row, column, and main diagonal has the required magic constant.

The magic constant for an order-n normal magic square is:

$$M = \frac{n(n^2 + 1)}{2}$$

For each number x, its complement is calculated as:

$$x' = n^2 + 1 - x$$

Thus, every number and its complement add to $n^2 + 1$. For a 4×4 square, the numbers range from 1 to 16, and complementary pairs sum to 17. For example, 1 pairs with 16, 2 with 15, and 8 with 9.

Construction procedure

1. Fill the $n \times n$ grid in row-major order with consecutive numbers from 1 to n^2.
2. Divide the grid conceptually into 4×4 blocks.
3. In each block, retain the numbers on the two main diagonals of that block.
4. Replace every other number with its complement $n^2 + 1 - x$.
5. Verify that each row, each column, and both main diagonals sum to M.

For example, begin a 4×4 grid as follows:

Table 2: Sequential arrangement used to construct a normal 4×4 square

1 2 3 4

5 6 7 8

9 10 11 12

1 2 3 4

13 14 15 16

Retaining the two diagonal patterns and complementing the remaining cells gives:

Table 3: Completed normal 4×4 magic square

1 15 14 4

12 6 7 9

8 10 11 5

13 3 2 16

Each row and column sums to 34, which agrees with $M = 4(4^2 + 1)/2 = 34$. The two main diagonals also sum to 34. The same complement pattern can be applied to larger orders divisible by 4.

This method is suitable for constructing **doubly-even** normal magic squares. It does not directly construct squares of odd order or singly-even order, which require different methods. In the website, this method provides a systematic way to generate and check supported doubly-even boards.

5.6 Singly-Even LUX/Strachey Method

A magic square has a **singly-even order** when its size is even but not divisible by four, such as 6×6 or 10×10. These orders cannot use the Siamese method, which applies to odd orders, or the direct complementary-cell method used for doubly-even orders such as 4×4 and 8×8. A partition-and-swap construction, commonly associated with Strachey, can be used instead.

For an order $n = 4k + 2$, let $m = n/2$, which is odd. First construct an odd-order magic square of size $m \times m$. Use that square to form four $m \times m$ blocks, each containing a shifted copy of the values from 1 through m^2. The shifts keep the values in the four blocks distinct and together cover every integer from 1 through n^2.

The four blocks are placed in a 2×2 arrangement: top-left, top-right, bottom-left, and bottom-right. The construction then swaps selected columns between the left and right halves of the top and bottom block rows. These exchanges are essential: they correct the row, column, and diagonal totals that would otherwise differ from the required magic constant. A small central adjustment is also made according to the construction rules for singly-even orders.

The resulting square should contain every integer from 1 to n^2 exactly once, and each row, column, and main diagonal should sum to

$$M(n) = \frac{n(n^2 + 1)}{2}.$$

For example, a 6×6 normal magic square uses the values 1 through 36 and has magic constant 111. The method is more involved than the odd-order and doubly-even constructions because it combines a smaller odd square with carefully selected swaps. In the project, the singly-even construction is implemented in the backend generation logic and its output can be checked against the normal-square rules before being used in gameplay.

5.7 Ramanujan Date-Based Magic Square

Srinivasa Ramanujan was born on 22 December 1887. The well-known 4×4 magic square associated with his birth date places the date components **DD = 22**, **MM = 12**, **CC = 18**, and **YY = 87** in the first row. Here, **CC** represents the first two digits of the year 1887, while **YY** represents its last two digits. The complete arrangement is:

Table 4: Ramanujan's date-based 4×4 magic square

22 12 18 87

88 17 9 25

10 24 89 16

19 86 23 11

Each row and column totals **139**. Both main diagonals also total 139:

- Main diagonal: 22 + 17 + 89 + 11 = **139**

- Opposite diagonal: 87 + 9 + 24 + 19 = **139**

Some other groups in this particular arrangement also total 139. The four corner entries sum to 22 + 87 + 19 + 11 = **139**, and the central 2×2 block sums to 17 + 9 + 24 + 89 = **139**. These properties make the square a useful example for studying how several groups of entries can share the same total.

This is a **date-based magic square**, rather than a normal 4×4 square containing each integer from 1 through 16 exactly once. Its entries were selected to relate to Ramanujan's birth date and satisfy the equal-sum properties above. Therefore, the normal-square complementary-pair property—where opposite pairs from 1 to 16 sum to 17—does not describe this date-based arrangement. Likewise, **1729** is associated with Ramanujan through a separate number-theory story; it is not an entry in the bottom row of this square.

In the project's date-based square feature, the user supplies a date and the date components are arranged to demonstrate the construction. The resulting entries and sums can be checked by adding each row, column, and main diagonal. The example above shows the specific square commonly associated with Ramanujan's birth date.

CHAPTER 6: SYSTEM REQUIREMENTS

6.1 Hardware Requirements

The project is a web-based application, so it does not require specialized hardware. Users need a device capable of running a modern web browser and a stable internet connection to access the hosted application.

Minimum hardware requirements:

- **Processor:** Dual-core processor or equivalent.
- **Memory:** 4 GB RAM.
- **Storage:** At least 200 MB of available space for browser files and temporary data.
- **Display:** Screen resolution of 1280 × 720 pixels or higher; the responsive interface can also be used on mobile screens.
- **Input devices:** Keyboard and mouse, or a touchscreen device.
- **Network:** Internet connection for accessing the hosted website and its online services.

Development and server requirements:

- **Processor:** Dual-core processor or better.
- **Memory:** 4 GB RAM or more.
- **Storage:** At least 1 GB of available space for the application, dependencies, and SQLite database.
- **Network:** Internet access for installation, deployment, and optional AI chatbot requests.

Actual resource use depends on the hosting provider, number of users, database size, and whether optional AI features are enabled.

6.2 Software Requirements

The software requirements for the Ramanujan's Magic Square project are listed below:

- **Operating system:** Windows, Linux, or macOS for development and deployment.
- **Frontend:** HTML5, CSS3, and JavaScript for the website interface and interactive gameplay.
- **Backend:** Python 3 with the Flask web framework to handle application routes, user accounts, gameplay, and chatbot requests.
- **Database:** SQLite to store user accounts, game attempts, sessions, achievements, and related application records.
- **Browser:** A modern web browser such as Google Chrome, Microsoft Edge, Firefox, or Safari with JavaScript enabled.
- **Python packages:** Flask and the other dependencies listed in the project's requirements.txt file.
- **Optional AI service:** An OpenAI API key configured on the server to enable AI-generated chatbot replies. Without the key, the chatbot uses its curated response system.
- **Development tools:** A code editor, Python environment, and terminal or command prompt for installing dependencies and running the application.
- **Network connection:** Required to access the hosted application and, when AI chat is enabled, to communicate with the AI service.
- **Persistent storage:** A persistent data directory for the SQLite database when the application is deployed, so user and gameplay records remain available after service restarts or updates.

6.3 Functional Requirements

The system shall provide the following functions:

- **User registration and login:** Users shall be able to create an account, log in, log out, and manage their profile.
- **Magic-square learning:** The system shall display explanations of magic-square rules, the magic constant, construction methods, and Ramanujan's 4×4 square.
- **Square generation:** The system shall generate valid magic squares for the supported board sizes.
- **Puzzle modes:** Users shall be able to play Swap, Fill, and Manual modes.
- **Input validation:** The system shall check puzzle entries and indicate whether the required rows, columns, and diagonals satisfy the target sum.
- **Hints and undo:** Users shall be able to request hints and undo supported moves during gameplay.
- **Daily Challenge:** The system shall provide a daily puzzle, record completion, and track challenge streaks.
- **Progress tracking:** The system shall record attempts and show relevant statistics, achievements, and game history on the dashboard.
- **Leaderboards:** The system shall display player rankings based on eligible puzzle results.
- **Chatbot assistance:** The system shall answer questions about MagicSquare and the website in supported reply languages. AI-based responses shall be available when the required API configuration is present.
- **Account data management:** Users shall be able to export their account data and use available profile and account-management functions.
- **Administrative functions:** Authorized administrators shall be able to access the system's available user, gameplay, and chatbot-review tools.

6.4 Non-Functional Requirements

The system should meet the following non-functional requirements to provide a dependable and accessible experience:

- **Usability:** The interface should be clear and easy to navigate, allowing users to understand the game controls, rules, and feedback with minimal effort.
- **Performance:** Pages and puzzle actions should respond promptly. Magic-square generation and validation should complete without unnecessary delays for supported board sizes.
- **Reliability:** The system should handle valid and invalid inputs consistently and provide clear messages when an operation cannot be completed.
- **Data persistence:** User accounts, achievements, and gameplay records should remain available across sessions and application updates when the database is hosted on persistent storage.
- **Security:** Passwords should be stored as secure hashes, protected pages should require authentication, and sensitive configuration values should be kept on the server.
- **Compatibility:** The website should work on current desktop and mobile browsers, with layouts that adjust to different screen sizes.
- **Maintainability:** The application should use organized frontend and backend modules so that features, game rules, and content can be updated without unnecessary changes to unrelated parts.
- **Accessibility:** Text, controls, and feedback should be readable and usable with common input methods, including keyboard navigation where supported.
- **Scalability:** The system should allow future improvements, such as additional learning content or database capacity, without requiring a complete redesign.
- **Privacy:** The application should handle account and gameplay data carefully. When AI chat is enabled, only the configured question, reply language, and relevant guide content should be sent to the AI service.

.

CHAPTER 7: SYSTEM ARCHITECTURE

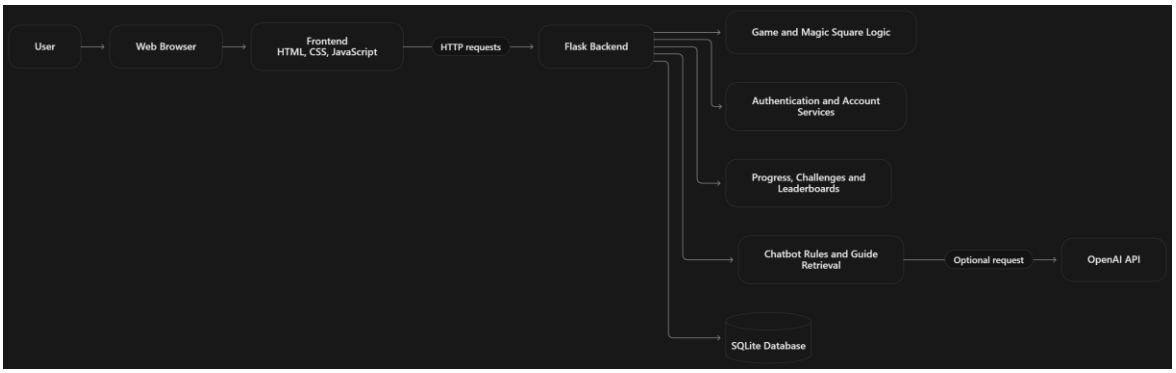
7.1 Overview of System Architecture

Ramanujan's Magic Square is implemented as a web application using a client–server architecture. The user interacts with the website through a browser. The frontend displays the pages, puzzle board, account forms, dashboard, and chatbot interface. It is built with HTML, CSS, and JavaScript.

The frontend communicates with a Python Flask backend through HTTP requests to application routes and APIs. The backend handles authentication, puzzle generation and validation, progress tracking, achievements, challenges, leaderboards, and chatbot requests. User and gameplay records are stored in an SQLite database.

The chatbot uses curated project knowledge for supported topics. When an OpenAI API key is configured, it can also use an AI service to generate answers based on the current question, selected reply language, and relevant guide excerpts. If AI is unavailable, the chatbot can use its curated responses as a fallback.

Figure 7.1: High-level system architecture. The diagram shows how the user's browser communicates with the Flask application, database, and optional AI service.



7.2 Architecture Components

The system contains the following main components:

- **User interface:** Provides the website pages and interactive controls. JavaScript sends requests to the backend and updates the displayed content.
- **Flask application:** Receives requests, applies application rules, checks authentication, and returns responses to the frontend.
- **Magic-square engine:** Constructs and validates squares. It checks whether the required numbers are used and whether the relevant rows, columns, and diagonals reach the target sum.
- **Puzzle gameplay services:** Support Swap, Fill, and Manual modes, including move tracking, hints, undo, timers, and attempt submission.

- **Account services:** Handle registration, login, logout, password changes, email-related workflows, data export, and account deletion.
- **Progress services:** Record attempts and calculate information used by the dashboard, achievements, Daily Challenge, and leaderboards.
- **Chatbot services:** Apply the site's topic scope and use curated rules and guide content. If configured, they can request a response from OpenAI.
- **SQLite database:** Persists account information and game-related records. The configured data directory must use persistent storage in a deployed environment if records are to survive service replacement or redeployment.

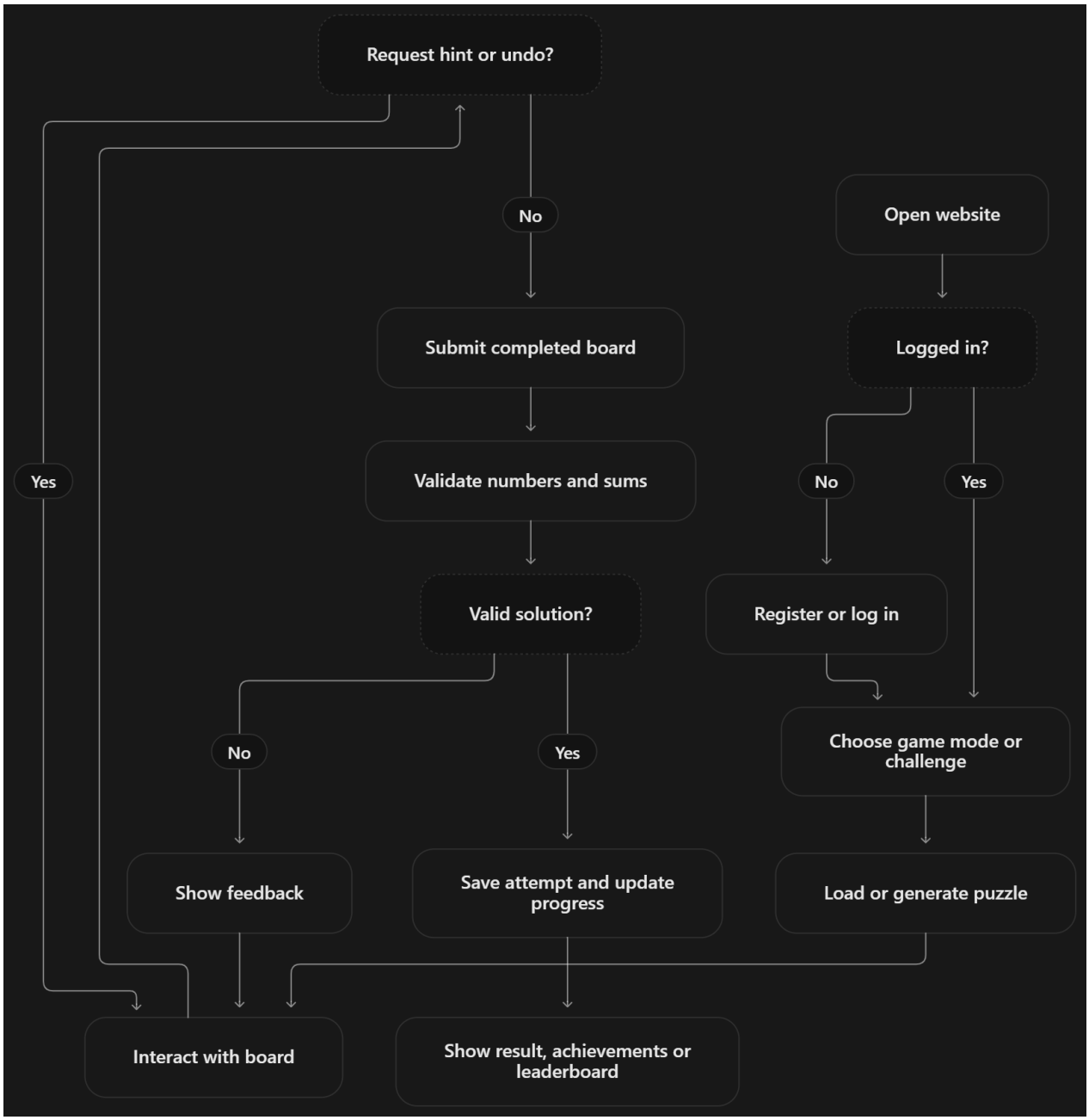
7.3 System Workflow

When a user opens the website, the browser loads the frontend pages and scripts. If the user signs in, the backend verifies the credentials and creates an authenticated session. The user can then start a puzzle, select a supported mode or challenge, and interact with the board.

During gameplay, the frontend sends relevant actions to the backend. The backend checks the request, applies the game rules, and returns a result. When a user submits a completed attempt, the system validates the square and records the result for progress and leaderboard features. Dashboard pages request the user's saved statistics and achievements.

For chatbot questions, the frontend submits the current question and selected language. The backend checks the question against the chatbot's scope and knowledge. It returns a curated response or, when configured, uses the optional AI service with relevant guide material. The response is displayed in the chat interface.

Figure 7.2: Typical gameplay workflow. The flow shows the sequence from starting a puzzle to validation and progress recording.

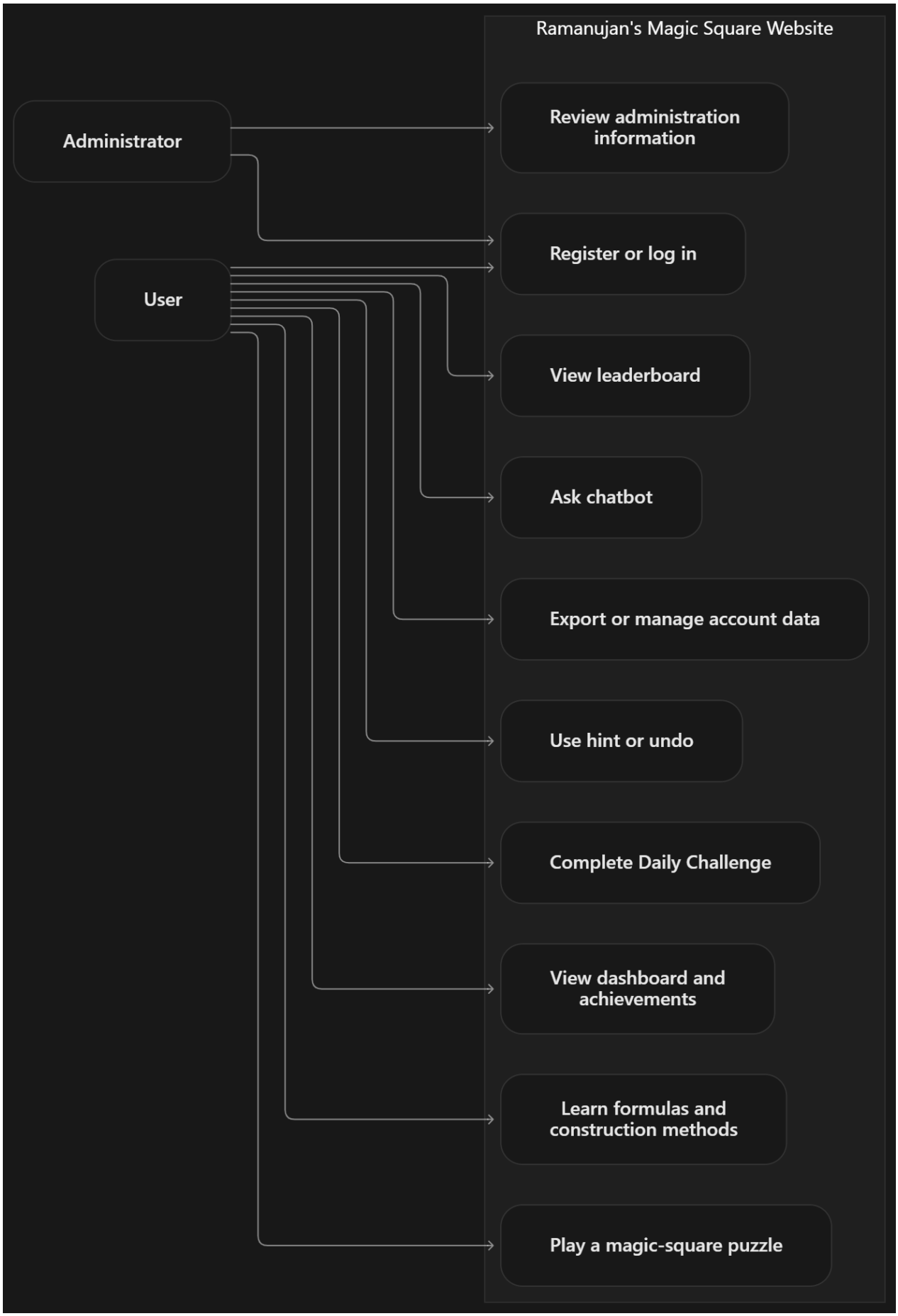

Request hint or undo?
No
Yes
Submit completed board
Validate numbers and sums
Valid solution?
No
Yes
Show feedback
Save attempt and update progress
Interact with board
Show result, achievements or leaderboard
Open website
Logged in?
No
Yes
Register or log in
Choose game mode or challenge
Load or generate puzzle

CHAPTER 8: SYSTEM DESIGN

8.1 Use Case Diagram

The primary actor is the **User**, who may use public pages, create an account, play puzzles, view progress, and ask the chatbot for help. An **Administrator** has additional access to administrative features for reviewing site information and chatbot feedback.

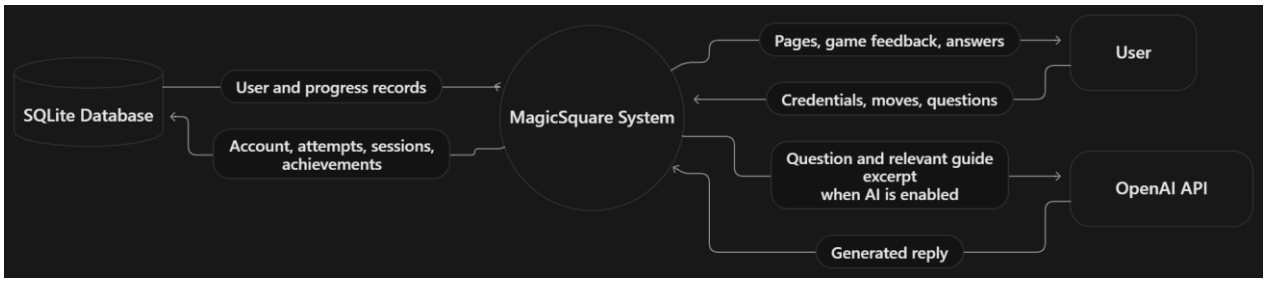
Figure 8.1: Main use cases. The diagram summarizes the actions available to users and administrators.


Ramanujan's Magic Square Website
Administrator
User
Review administration information
Register or log in
View leaderboard
Ask chatbot
Export or manage account data
Use hint or undo
Complete Daily Challenge
View dashboard and achievements
Learn formulas and construction methods
Play a magic-square puzzle

8.2 Data Flow Diagram

At the highest level, the user provides account details, puzzle actions, and chatbot questions to the website. The system returns pages, validation feedback, progress information, and chatbot answers. Account and game records are stored in the database.

Figure 8.2: Context-level data flow diagram. The diagram identifies the main information exchanged between users, the application, and stored data.



The user's credentials are processed by the authentication component. Puzzle actions are sent to the game component, which checks the board and returns feedback. Successful and eligible attempts are saved so that progress features can retrieve them later. Chatbot requests are checked against the website's gaming scope before the system produces a response.

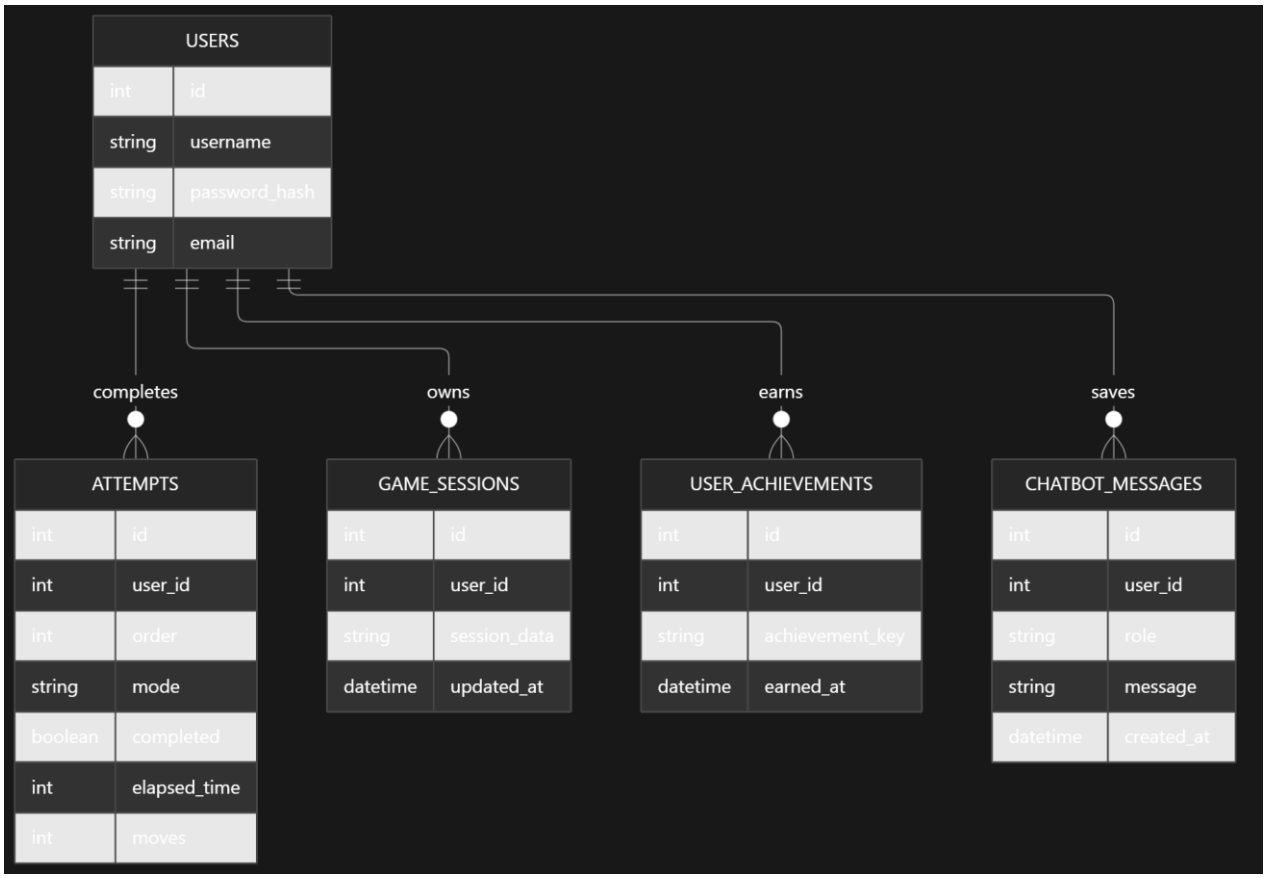
8.3 Database Design

The application uses SQLite to store persistent records. The main data entities include users, attempts, game sessions, achievements, and chatbot messages. These records support account access, gameplay history, progress summaries, and chat history.

- **Users:** Stores account identifiers and user-related fields such as username, password hash, and profile or email information. Passwords should be stored as hashes rather than plain text.
- **Attempts:** Stores puzzle attempt information, such as the user, puzzle order, mode, result, time, moves, and relevant challenge details.
- **Game sessions:** Stores information needed to resume or manage an active puzzle session.
- **User achievements:** Records achievements earned by users and the associated achievement identifiers.
- **Chatbot messages:** Stores chatbot conversation records for the application's chat-history feature.

A user can have multiple attempts, game sessions, achievements, and chatbot messages. Each of those records is associated with a user through a user identifier. The exact columns and constraints are defined by the application's database initialization and migration code.

Figure 8.3: Simplified database relationship diagram. A single user may be associated with many gameplay and chatbot records.


USERS
int id
string username
string password_hash
string email
completes
owns
earns
saves
ATTEMPTS
int id
int user_id
int order
string mode
boolean completed
int elapsed_time
int moves
GAME_SESSIONS
int id
int user_id
string session_data
datetime updated_at
USER_ACHIEVEMENTS
int id
int user_id
string achievement_key
datetime earned_at
CHATBOT_MESSAGES
int id
int user_id
string role
string message
datetime created_at

8.4 User Interface Design

The interface is organized around the activities users perform most often: learning about magic squares, playing puzzles, tracking progress, and getting help. Navigation provides access to the game, Daily Challenge, Manual mode, dashboard, leaderboard, formula material, and profile functions.

The puzzle interface presents the board and the controls needed for the selected mode. In Swap mode, users rearrange tiles; in Fill mode, they enter values into the grid; and Manual mode supports practice. Hints and undo are available as gameplay controls. The system displays validation feedback after a submission so users can understand whether the board meets the rules.

The dashboard presents saved performance information, including attempts and achievements. The chatbot appears as an in-site help feature and supports English and Kannada replies. The interface is designed to work on desktop and mobile screens, with layouts adapting to narrower viewports.

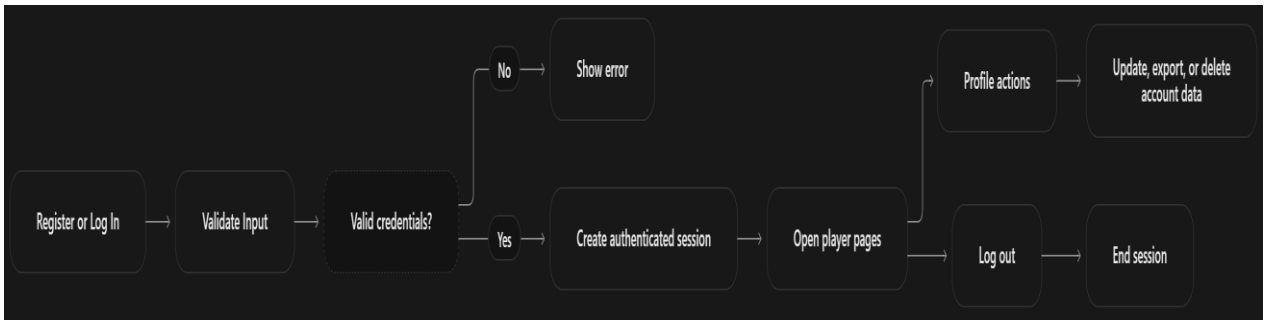
CHAPTER 9: MODULE DESCRIPTION

9.1 User Authentication and Profile Module

This module manages account access and profile-related actions. A new user can register with an account, and an existing user can log in and log out. The backend validates requests and uses password hashing so that passwords are not stored as plain text. Authenticated users can access their profile and permitted account functions, such as changing a password, managing email details, exporting account data, or deleting an account.

The module maintains the user's authenticated session and supplies the identity required by other features. For example, the gameplay and dashboard modules use the signed-in user's identifier to associate attempts, achievements, and progress with the correct account. Account records are stored in the users table.

Figure 9.2: Authentication and profile flow. This diagram outlines the main account workflow.



9.2 Magic Square Generation Engine

The generation engine creates valid squares for supported orders from 3×3 through 10×10. A magic square of order n contains n^2 cells, and a normal magic square uses each integer from 1 to n^2 exactly once. Its target magic constant is:

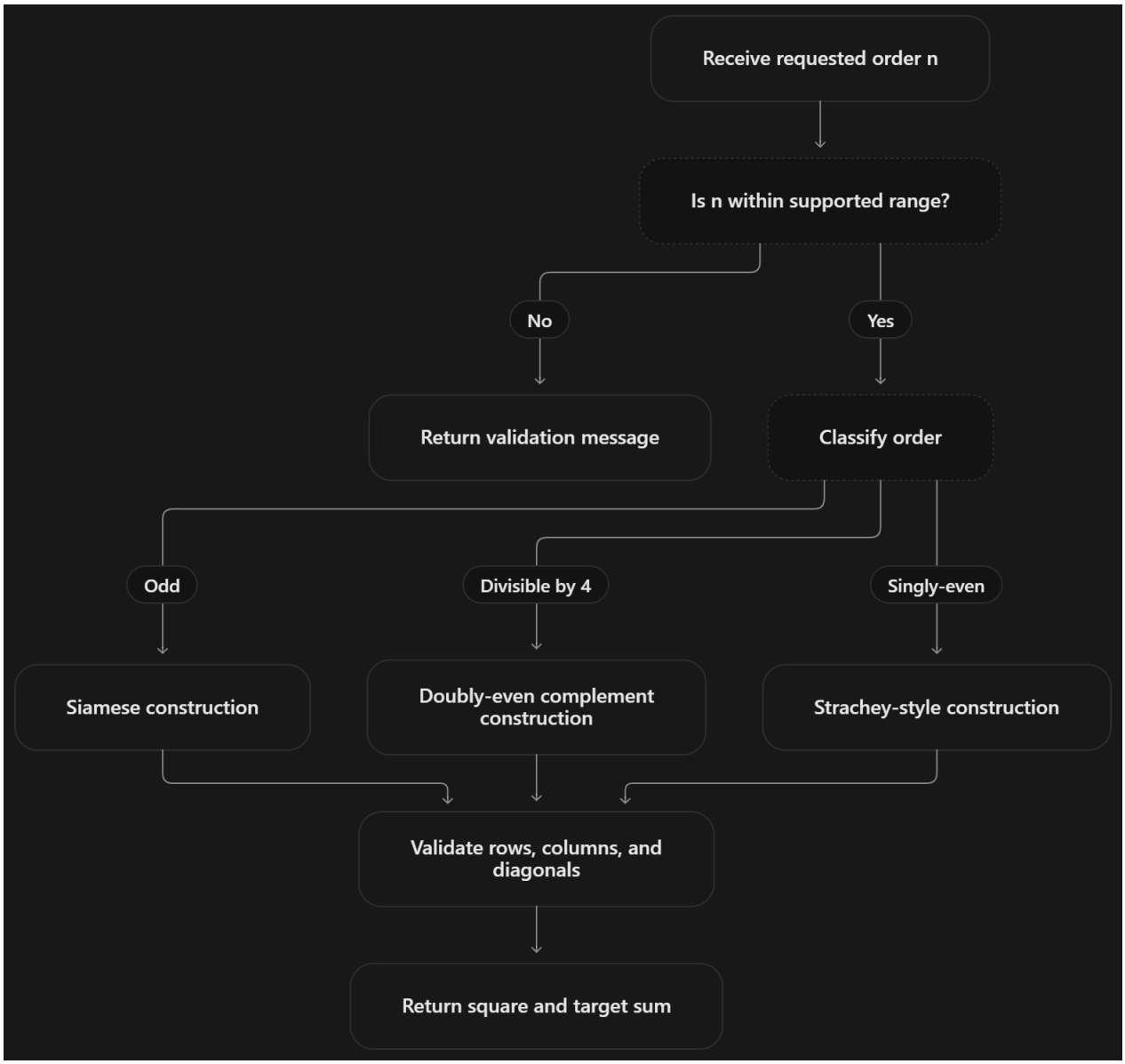
$$M(n) = \frac{n(n^2 + 1)}{2}$$

The engine selects a construction approach based on the order. Odd orders use the Siamese method; doubly-even orders, which are divisible by four, use a complementary-number method; and singly-even orders use a Strachey-style construction. The generated square can then be checked by the application's validation logic.

The website also includes the date-based 4×4 square associated with Ramanujan. This is a special square built from shifted date components, rather than a normal square containing only the numbers 1 through 16. Its entries are evaluated against its own target total.

Figure 9.3: Square generation flow. The order determines which construction method the engine applies.

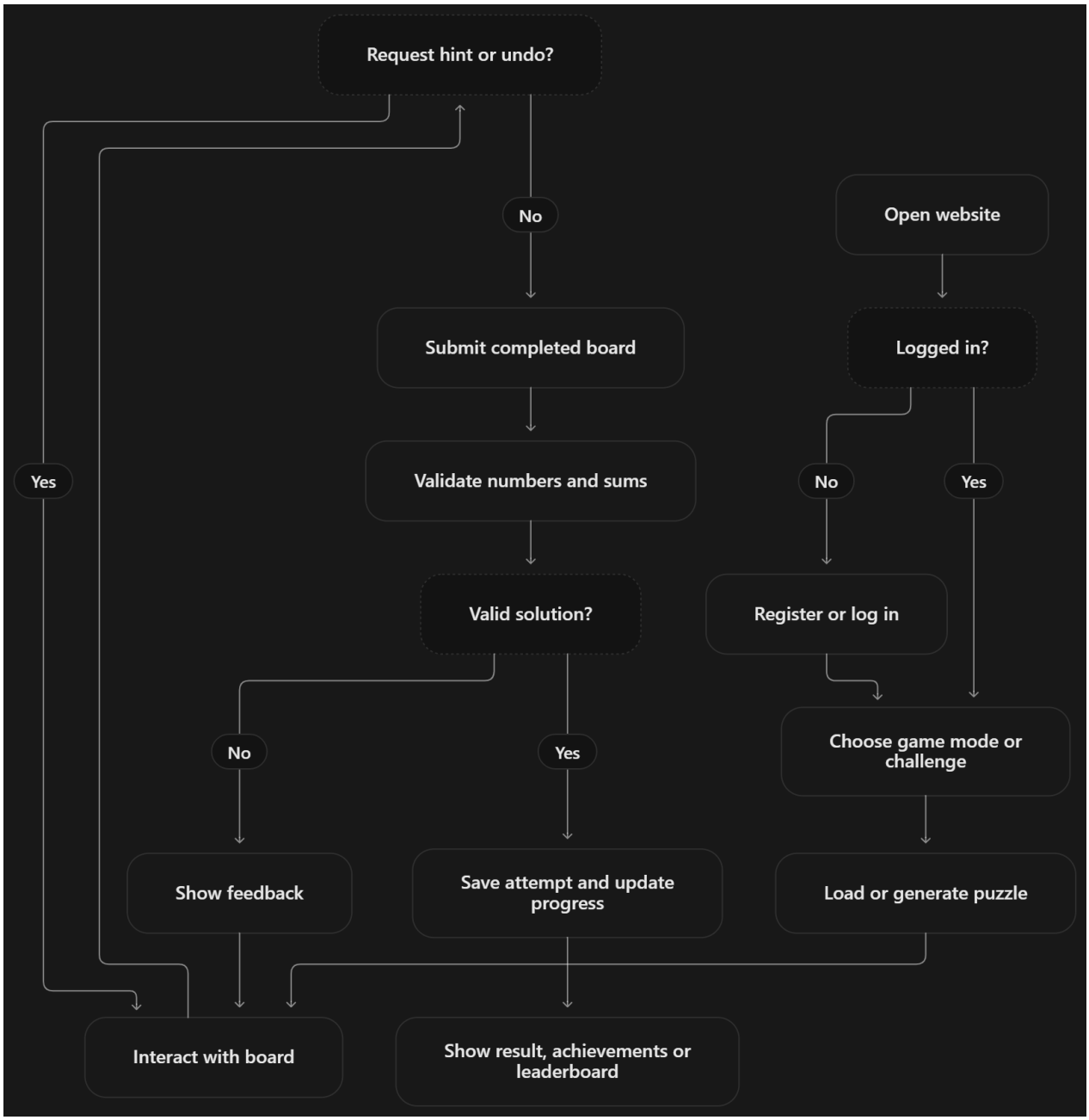
.



9.3 Puzzle Gameplay Module

The gameplay module presents generated or prepared squares as interactive puzzles. It supports Swap, Fill, and Manual modes. In Swap mode, the player rearranges a scrambled board by exchanging tiles. In Fill mode, the player enters values into empty cells and submits the board for checking. Manual mode provides a practice space for working with a square without treating it as a scored challenge.

During a round, the interface displays the board and relevant game information, such as elapsed time, moves, and sums. On submission, the application checks that the entries satisfy the rules for the selected puzzle. Completed attempts can be saved against the player’s account for later review.



9.4 Hint and Undo Module

The Hint feature assists a player who needs help identifying a useful next step. It provides guidance according to the current game mode and puzzle state. In timed play, using a hint applies the game's documented time penalty of 15 seconds. The hint does not replace the player's full solution; the player continues interacting with the board.

Undo allows a player to reverse a recent action, such as a tile swap or entry change, where supported by the active mode. The interface maintains the necessary move state so the previous board arrangement can be restored. These tools support experimentation while keeping the player responsible for completing the puzzle.

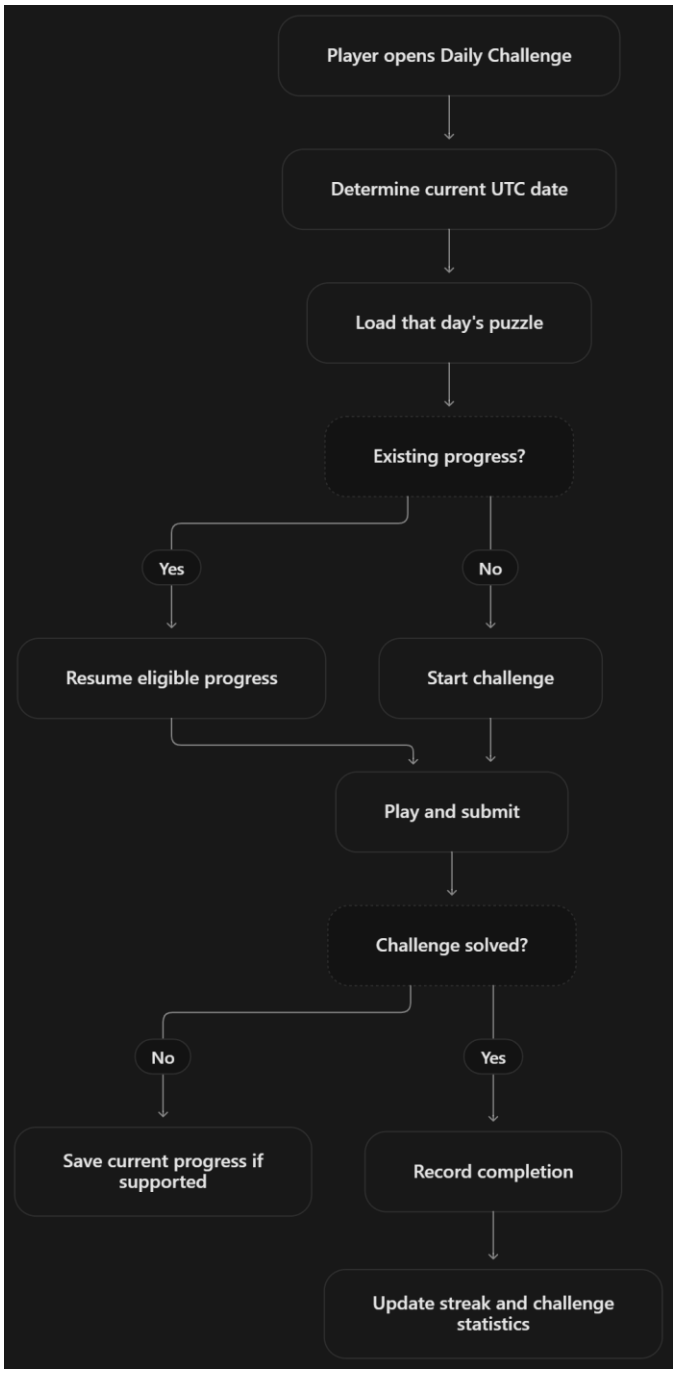
Ramanujan's date square is one particular non-normal arrangement, not a representative of all normal 4 × 4 squares. Counting normal squares does not directly count date-encoded squares. The latter problem has a different value set and often fixed first-row data, so it needs a separately stated model.

9.5 Daily Challenge Module

The Daily Challenge presents a shared daily puzzle experience. The challenge changes according to the application's daily schedule, which is based on midnight UTC. The interface checks the player's progress for the current challenge and records a completed result when the puzzle is solved.

The module also tracks consecutive challenge participation as a streak. If a user misses a day, the streak rules apply to the recorded dates; the dashboard displays the resulting progress. Challenge eligibility and board order follow the game's progression rules.

Figure 9.5: Daily Challenge flow.



9.6 Leaderboard Module

The leaderboard retrieves eligible completed results and orders them according to the game's ranking rules. It allows players to compare their performance for supported square orders and, where available, daily challenges. Results are associated with player accounts so that the site can display names and relevant performance details.

The leaderboard depends on saved attempt data. A result that does not meet the applicable eligibility rules is not included. This module reads the recorded results and presents them; it does not generate or alter puzzle solutions.

9.7 Dashboard and Achievement Module

The dashboard summarizes a player's activity and progress. It can display attempt history, statistics, records, challenge streaks, and unlocked or locked achievements. The achievement module compares recorded activity with defined criteria, such as solving a square, completing different orders, reaching an attempt milestone, or maintaining a challenge streak.

When a qualifying action is recorded, the system can update the user's achievement status. Dashboard information is retrieved for the signed-in user, keeping personal progress separate between accounts. User achievements are stored in the user_achievements table, while gameplay results are drawn from saved attempts and sessions.

9.8 Formula and Learning Module

This module presents explanations that help users understand the mathematics behind the puzzles. It covers the definition of a magic square, the magic constant formula, supported board sizes, construction methods, and the special date-based square associated with Ramanujan. The formula section connects the displayed target sum to the order n of a normal square.

For example, a normal 4×4 square has target sum:

$$M(4) = \frac{4(4^2 + 1)}{2} = \frac{4 \cdot 17}{2} = 34$$

This normal 4×4 square uses the numbers 1 through 16. Ramanujan's date-based square is a different construction with shifted values and a target sum of 139. The module explains this distinction so users do not confuse the two kinds of square.

9.9 Admin Module

The admin module provides authorized administrative views for managing and reviewing selected application data. Depending on the permissions configured for the account, an administrator can

review user or attempt information and inspect chatbot feedback or reports. Access to these functions is restricted to authorized users.

Administrative actions operate through backend routes and database queries. The module is separate from ordinary gameplay: players use the public game and dashboard features, while administrators use the available review tools to support application management.

The diagram syntax above can be rendered as figures for the report. Keep each caption with its diagram, and refer to the figure number in the surrounding text.

CHAPTER 10: DATABASE DESIGN AND DESCRIPTION

10.1 Users Table

The users table stores account, authentication, and profile information. Passwords are stored as hashes rather than plain text. The table also contains fields for administrator status, email verification, password-reset tokens, daily streaks, and the chatbot reply language.

Table 5: Users table schema

Field	Type	Description
id	INTEGER	Primary key; uniquely identifies each user.
username	TEXT	Required, unique account name.
password_hash	TEXT	Required hashed password.
is_admin	INTEGER	Indicates administrator access; defaults to 0.
created_at	TEXT	Account creation time; defaults to the current time.
daily_streak	INTEGER	Current Daily Challenge streak; defaults to 0.
last_daily_date	TEXT	Date of the user's most recent Daily Challenge completion.
email	TEXT	Optional email address; unique when provided.
email_verified	INTEGER	Indicates whether the email address is verified; defaults to 0.
verify_token	TEXT	Token used in the email verification workflow.
verify_token_expires	INTEGER	Expiry time for the verification token.
reset_token	TEXT	Token used in the password reset workflow.
reset_token_expires	INTEGER	Expiry time for the password reset token.
chatbot_language	TEXT	Preferred chatbot reply language; defaults to English (en).

10.2 Attempts table

The attempts table stores records of completed puzzle attempts. These records support attempt history, personal statistics, leaderboard results, and achievement checks. Each attempt belongs to one user.

Table 6: Attempts table schema

Field	Type	Description

Field	Type	Description
id	INTEGER	Primary key; uniquely identifies an attempt.
user_id	INTEGER	Foreign key referencing users.id.
square_order	INTEGER	Order of the puzzle, such as 3 for a 3×3 square.
moves_made	INTEGER	Number of moves used during the attempt.
time_taken	INTEGER	Time taken to complete the attempt.
hints_used	INTEGER	Number of hints used; defaults to 0.
is_daily	INTEGER	Indicates whether the attempt was a Daily Challenge; defaults to 0.
challenge_date	TEXT	Date associated with the Daily Challenge, when applicable.
date_attempted	TEXT	Time the attempt was recorded; defaults to the current time.
mode	TEXT	Game mode used, such as Swap or Fill.

10.3 Game Sessions Table

The game_sessions table stores information about games that are in progress or have been completed. It allows the application to track the generated solution, moves, hints, and Fill-mode checks for a session.

Figure 9. User-supplied handmade model of the 139 square

Table 7: Game sessions table schema

Field	Type	Description
id	TEXT	Primary key; unique identifier for the game session.
user_id	INTEGER	Foreign key referencing users.id.
square_order	INTEGER	Order of the magic square being played.
solved_grid	TEXT	Serialized representation of the solution grid.
is_daily	INTEGER	Indicates whether the session is a Daily Challenge; defaults to 0.
challenge_date	TEXT	Date of the associated Daily Challenge, when applicable.
started_at	INTEGER	Session start time recorded as a timestamp in milliseconds.
moves_made	INTEGER	Number of moves made; defaults to 0.
hints_used	INTEGER	Number of hints used; defaults to 0.
completed_at	INTEGER	Completion timestamp; remains empty while the session is active.
mode	TEXT	Game mode; defaults to scramble for existing or newly created records without a specified mode.
attempts_used	INTEGER	Number of checks used in Fill mode; defaults to 0.

10.4 User Achievements Table

The user_achievements table records the achievements earned by each user. A user can earn an achievement only once because the combination of user_id and achievement_key is unique.

Table 8: User achievements table schema

Field	Type	Description
id	INTEGER	Primary key; uniquely identifies an achievement record.
user_id	INTEGER	Foreign key referencing users.id.
achievement_key	TEXT	Identifier for the achievement earned.
earned_at	TEXT	Time the achievement was earned; defaults to the current time.

Chapter 11: System Implementation

11.1 Dependency and Library Setup

The application's active backend is implemented in Python using the Flask web framework. Its main dependencies are listed in requirements.txt: Flask, which handles HTTP requests, routes, sessions, and JSON responses, and bcrypt, which securely hashes and checks user passwords. SQLite is accessed through Python's built-in sqlite3 module, so a separate database server or database driver package is not required. Other built-in Python modules support tasks such as file handling, JSON processing, CSV export, time management, and HTTP requests to the optional AI service.

The website's frontend uses HTML, CSS, and JavaScript files served from the public directory. These files do not require a separate frontend framework or build step for the current Flask deployment. The AI chatbot can use the OpenAI API when the server is configured with an OPENAI_API_KEY; the application sends API requests using Python's built-in HTTP facilities, so no OpenAI Python SDK is required.

The Python dependencies are installed from the project root with:

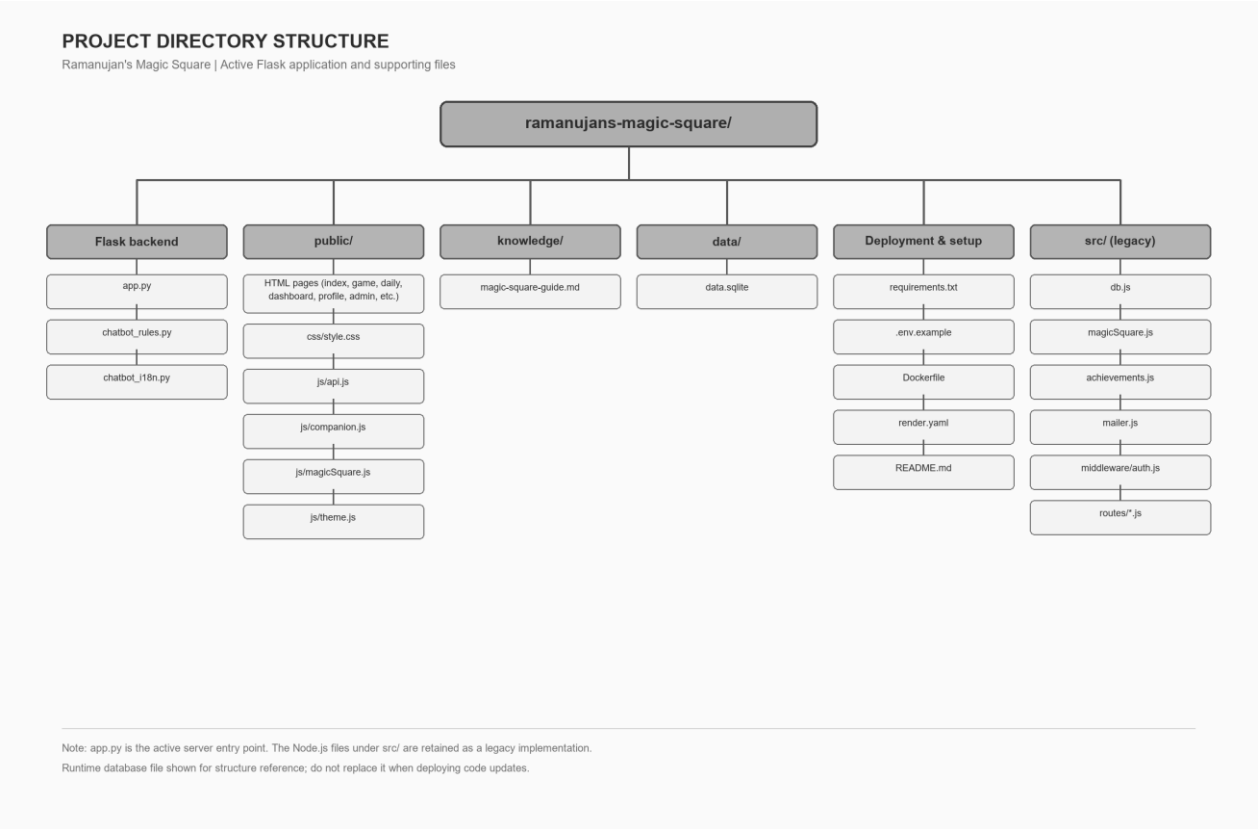
python -m pip install -r requirements.txt

The application can then be started locally with:

python app.py

The project also contains a package.json with Node.js and Express-related packages from an alternate or legacy setup. The included Dockerfile, however, installs Python requirements and starts app.py, so the documented deployment path uses Flask. These Node.js packages are not needed to run that Flask deployment.

11.2 Project Folder Structure



11.3 Frontend Implementation

The frontend of Ramanujan's Magic Square is built with HTML, CSS, and JavaScript. HTML provides the structure of the website, CSS controls the layout and visual appearance, and JavaScript handles user interactions and communicates with the backend. The interface is designed to work on desktop and mobile screens, with responsive layouts for navigation, game boards, forms, and dashboard content.

The frontend is organized into pages for the main game, Daily Challenge, Manual mode, dashboard, leaderboard, formula and learning content, profile management, and administration. Shared styling and JavaScript files support a consistent appearance and common behavior across these pages. The game interface displays the selected square, its numbers, relevant sums, timer, move count, and available controls.

JavaScript manages the interactive parts of gameplay. In Swap mode, it lets the player select and exchange tiles while tracking moves and time. In Fill mode, it accepts entries into an initially blank grid and submits them for validation. Manual mode provides a space for practice without treating the activity as a scored attempt. The interface also handles hints, undo, board-size selection, completion feedback, and Daily Challenge interactions.

The frontend uses HTTP requests to exchange data with the Flask backend. It sends requests for actions such as logging in, starting or checking a game, loading attempt history, retrieving dashboard information, and submitting feedback to the chatbot. Responses are used to update the visible page without requiring the user to manage database operations directly. Account and administrative pages similarly use backend endpoints for authorized tasks.

The chatbot interface accepts typed questions and lets users select English or Kannada replies. It sends the question and selected language to the backend, which applies the project's topic rules and configured answer service. Voice chat and screenshot analysis are not part of the current frontend. User-facing messages indicate the available features and provide feedback when an action succeeds or requires attention.

11.4 Backend Implementation

The backend of Ramanujan's Magic Square is developed using Python and the Flask web framework. Flask receives requests from the browser, applies the required application logic, communicates with the SQLite database, and returns responses to the frontend. The backend serves the website pages and provides API endpoints for account management, gameplay, progress tracking, leaderboards, achievements, and chatbot requests.

User-related routes support registration, login, logout, password changes, email management, account data export, and account deletion. Passwords are stored as secure hashes rather than as plain text. Session-based authentication is used to identify logged-in users and protect routes that require an account. Configuration values such as the session secret and optional AI service key are supplied through environment variables.

Gameplay endpoints handle starting a puzzle, creating a new square, checking a completed board, recording progress, and retrieving attempt history. The server-side game logic generates and validates magic squares. For a normal square of order n, validation checks that the grid contains the required values from 1 to n^2, without duplicates, and that every row, column, and main diagonal has the target sum $M = n(n^2 + 1)/2$. Daily Challenge endpoints provide the day's puzzle and its progress, while leaderboard endpoints return eligible results for comparison.

The backend also stores user achievements and game-session information in SQLite. Database initialization creates the required tables when the application starts and applies necessary schema updates. The DATA_DIR environment variable can point the application to persistent storage so that account and gameplay records remain available after application restarts or code updates, provided the database file and storage are retained.

Chatbot requests are checked against the project's MagicSquare and gaming scope. Curated rules and guide content provide responses when an AI service is not configured or unavailable. When AI mode is enabled, the backend sends the current question, selected reply language, and relevant guide excerpts to the configured service; usernames, saved chat history, and live board data are excluded. This server-side design keeps the API key out of the browser and allows the chatbot to use a fallback response path.

6:27 PM

11.5 Database Implementation

The project uses SQLite to store account information and game-related data. SQLite was selected because it is lightweight, does not require a separate database server, and is suitable for a web application of this size. The Flask backend connects to the database and performs operations such as creating accounts, retrieving player records, saving attempts, and updating achievements. The database file is stored in the application's data directory, which can be configured through the DATA_DIR environment variable. For a deployed application, this directory must point to persistent storage so that user accounts and progress remain available after code updates or service restarts.

The database contains tables for users, attempts, game sessions, user achievements, and chatbot messages. The users table stores account details and password hashes. The attempts table records puzzle activity and results, while game sessions preserve the state needed to resume supported unfinished rounds. The user achievements table records badges earned by each player, and the chatbot messages table stores chat records associated with a user. These tables are linked through user identifiers so that player-specific records can be retrieved and managed together. The application creates required tables and applies schema updates when it starts, allowing the database structure to evolve as features are added. Passwords are stored as hashes rather than plain text, and account-management functions allow users to export or delete their associated data.

11.6 Algorithm Implementation

The system uses algorithms to generate valid magic squares, create puzzles, check player solutions, and calculate progress. The construction method depends on the order of the square. For an order n, a normal magic square contains each integer from 1 through n^2 once, and its magic constant is:

$$M = \frac{n(n^2 + 1)}{2}$$

The application supports orders 3×3 through 10×10. Odd orders use the Siamese method, orders divisible by four use the doubly-even complement method, and singly-even orders use a Strachey-style method. After construction, the square is checked to confirm that every row, column, and main diagonal equals M.

Algorithm 1: Generate a Magic Square

INPUT: Order n

OUTPUT: A valid n × n magic square

1. Check that n is within the supported range.

2. If n is odd:

   Generate the square using the Siamese method.

Else if n is divisible by 4:

Generate the square using the doubly-even complement method.

Else:

Generate the square using the singly-even Strachey method.

3. Calculate the expected magic constant:

M = n × (n² + 1) / 2

4. Validate every row, column, and main diagonal against M.

5. Return the square if all checks pass; otherwise report an error.

In the Siamese method, place 1 in the middle cell of the top row. For each following number, move one cell up and one cell right, wrapping around the edges when needed. If the destination is already occupied, move one cell down from the previous position instead. Continue until all numbers from 1 to n^2 have been placed.

For a doubly-even square, where n is divisible by four, first fill the grid in order from 1 to n^2. Then identify cells according to a repeating 4×4 pattern. Keep selected values unchanged and replace the others with their complements relative to $n^2 + 1$. This produces equal row, column, and diagonal totals.

For a singly-even square, where n is even but not divisible by four, divide the order into four smaller odd-order squares. Arrange copies of the smaller square in four quadrants, then exchange selected columns and cells between the quadrants according to the Strachey construction pattern. These exchanges preserve the required total while producing a valid square of the requested order.

Algorithm 2: Validate a Magic Square

INPUT: n × n grid G and expected magic constant M

OUTPUT: Valid or invalid, with the lines that fail

1. Confirm the grid has n rows and n columns.

2. Confirm every required number occurs exactly once.

3. For each row, calculate the sum of its cells.

4. For each column, calculate the sum of its cells.

5. Calculate the sums of the two main diagonals.

6. Compare each sum with M.

7. Return valid only if all checks pass.

In Swap mode, the application begins with a generated valid square and shuffles its values to create a puzzle. The player rearranges the tiles, and the validation algorithm checks whether the current arrangement restores the target square. In Fill mode, the player enters values into a blank grid; the

system checks the entries and reports which rows, columns, or diagonals meet the target. Manual mode allows practice without treating the result as a scored attempt.

The validation logic can be expressed mathematically. For grid entries a_{ij}, each row must satisfy $\sum_{j=1}^{n} a_{ij} = M$, each column must satisfy $\sum_{i=1}^{n} a_{ij} = M$, and the two main diagonals must also sum to M. For a normal square, the values must additionally be exactly the integers $1,2,\ldots,n^2$, with no duplicates.

11.7 Integration of Modules

The modules of Ramanujan's Magic Square are integrated through the Flask backend, the web interface, and the SQLite database. The frontend sends requests to the backend when a user registers, logs in, starts a puzzle, submits an answer, requests a hint, or opens a dashboard feature. The backend validates each request, applies the required game or account logic, and returns a response for the interface to display.

The authentication module works with profile and account features to identify the current user and protect their data. When a user plays a puzzle, the generation engine creates or loads a board and the gameplay module displays it. Validation checks the player's entries against the magic-square rules. Depending on the result, the application updates the game state, timer, move count, and attempt history. Hints and undo operate on the active game state while preserving the rules of the selected mode.

Daily Challenges use the square-generation and gameplay modules with a challenge date and supported board order. After completion, relevant results can contribute to progress records, achievements, and leaderboard entries. The dashboard retrieves stored statistics, attempts, and achievements for the logged-in user. The Formula and Learning section presents explanations alongside the mathematical principles used by the generation and validation logic.

The chatbot is integrated with the website's help interface and its curated guide. When AI service settings are configured, it can use the current question, selected reply language, and relevant guide excerpts to prepare a response; otherwise, it uses available rule-based answers. Each module communicates through defined routes and shared data structures, allowing the application to provide a consistent experience across gameplay, learning, and account features.

CHAPTER 12: TESTING AND VALIDATION

12.1 Test Cases and diagrams

The following test cases are designed to check the main functions of the Ramanujan's Magic Square application. Each case describes an action and the result expected from the system. Since these tests have not been executed as part of this response, the actual result and status should be filled in after testing the application.

Table 9: Functional test cases and expected outcomes

Test ID	Test case	Expected result	Actual result / status
TC-01	Register with valid account details	The account is created and the user can log in.	To be recorded
TC-02	Log in with valid credentials	The user is authenticated and can access their dashboard.	To be recorded
TC-03	Log in with an incorrect password	The system rejects the login and displays an appropriate message.	To be recorded
TC-04	Generate a 3×3 magic square	The generated square uses the required numbers and each row, column, and main diagonal sums to 15.	To be recorded
TC-05	Generate a square of another supported order	The square is generated, and its required line sums match the calculated magic constant.	To be recorded
TC-06	Enter duplicate or invalid numbers in a puzzle	The system identifies the invalid entry and does not mark the puzzle as correctly solved.	To be recorded
TC-07	Use the Hint and Undo controls	The hint provides the intended assistance, and Undo reverses the most recent supported move.	To be recorded
TC-08	Complete a Daily Challenge	The result is recorded according to the challenge rules and appears in the relevant progress view.	To be recorded
TC-09	View achievements and leaderboard	The system displays the user's earned achievements and applicable leaderboard records.	To be recorded
TC-10	Export account data	A CSV file downloads with appropriate headers and the user's available data.	To be recorded
TC-11	Ask the chatbot an in-scope question	The chatbot responds with relevant MagicSquare or website guidance, using the configured language and response mode.	To be recorded
TC-12	Ask the chatbot an off-topic question	The chatbot returns the project's configured scope-refusal response.	To be recorded

CHAPTER 13: RESULTS & OBSERVATIONS

13.1 Results

The project provides a web application for learning and playing magic-square puzzles. It supports square orders from 3×3 through 10×10, with construction methods selected according to the order. A generated normal magic square uses each number from 1 to n^2 once, and its rows, columns, and main diagonals have the target sum $M = n(n^2 + 1)/2$.

The application includes Swap, Fill, and Manual modes, together with hints, undo, timers, and puzzle validation. It also provides Daily Challenges, a leaderboard, achievements, player progress, and account-data export. The learning section and chatbot provide information about magic squares and the website. AI-powered chatbot answers require the server to be configured with an OpenAI API key; otherwise, curated responses are used.

Ramanujan's date-based 4×4 square is presented as a separate example. Its rows, columns, main diagonals, central 2×2 block, and four corners sum to 139. The project uses SQLite to store user and gameplay data.

13.2 Observations

- The magic-constant formula gives a consistent target for each supported order. For example, a 3×3 square has a target sum of 15, while a 4×4 square has a target sum of 34.
- The number 17 is the complementary-pair sum $n^2 + 1$ for a normal 4×4 square using the numbers 1 through 16. It should not be treated as a general property of Ramanujan's shifted date-based square.
- Ramanujan's date-based square has a constant of 139; 1729 is not an entry in its bottom row. The number 1729 is associated with a separate story about Ramanujan and sums of cubes.
- The different gameplay modes allow users to practise by rearranging a board, filling empty cells, or working manually.
- Hints and undo provide assistance during play, while timers, achievements, and leaderboards add progress and performance features.
- User records are stored in SQLite. Keeping those records across deployments depends on configuring persistent storage for the database.
- The chatbot can use curated responses without an API key. Its AI response mode depends on external API access and server configuration.

13.3 Result summary

Table 10: Project capability summary

Parameter	Result
Supported normal-square orders	3×3 through 10×10
Magic constant	$M = n(n^2 + 1)/2$
Ramanujan date-square line and selected-group sum	139
Gameplay modes	Swap, Fill, and Manual
Progress features	Daily Challenges, achievements, history, and leaderboards
Data storage	SQLite
Chatbot	Curated responses; AI responses when configured
Performance measurements	Not reported; require execution and measurement

These results describe the implemented features and mathematical behavior. Runtime reliability, response time, and user-experience claims should be added only after the corresponding tests have been run and recorded.

CHAPTER 14: LIMITATIONS

The project provides an interactive way to learn and practise magic squares, but it has several limitations:

- The application focuses on magic-square puzzles and related website guidance; it does not support unrelated games or general-purpose chat.
- The chatbot's AI responses depend on an OpenAI API key, internet access, and service availability. Without these, it uses curated responses, which may not cover every question.
- The chatbot provides guidance based on the project's available information and does not have access to the player's live board state for detailed, cell-by-cell analysis.
- The application does not include live multiplayer matchmaking or real-time team coordination.
- Voice chat and screenshot analysis are not currently available.
- Gameplay progress depends on the application being able to save data to its SQLite database. A deployment without persistent storage may lose user records when its storage is reset.
- The quality of the user experience depends on the device, browser, screen size, and internet connection.
- The current project does not include detailed performance measurements from large-scale or long-duration testing.
- The system's supported construction methods and board sizes are limited to those implemented in the application.
- The chatbot may misunderstand unclear or misspelled questions, and its responses should be checked when mathematical detail is important.

FUTURE ENHANCEMENTS

The project can be extended in several ways to improve its learning value, accessibility, and reliability:

- **Adaptive difficulty:** Adjust puzzle difficulty based on the player's progress, solving time, and use of hints.
- **Step-by-step solving guidance:** Explain how a move affects row, column, and diagonal totals, helping learners understand the reasoning behind each solution.
- **Expanded multilingual support:** Add more interface and chatbot languages while keeping the mathematical explanations consistent.
- **Improved chatbot knowledge:** Add reviewed guides and more detailed answers about construction methods, strategies, and common mistakes. AI responses can remain optional, with curated answers available as a fallback.
- **Teacher and classroom features:** Provide class groups, assigned puzzles, and progress summaries that teachers can use to support learning.
- **Additional puzzle types:** Introduce themed challenges, timed tournaments, and new variations while clearly distinguishing them from normal magic squares.
- **Accessibility improvements:** Add keyboard-friendly controls, screen-reader labels, adjustable text size, and high-contrast themes.
- **More detailed progress reports:** Show trends in solving time, accuracy, puzzle size, and hint usage, with downloadable summaries.
- **Expanded testing:** Automate tests for square generation, validation, account handling, data export, and chatbot scope behavior.
- **Deployment and backup improvements:** Document reliable database backup and restoration procedures so user accounts and progress remain available during hosting changes.

These enhancements should be introduced incrementally and tested before release. Any new feature that uses external AI or stores additional personal data should also explain what information is processed and how it is protected.

CONCLUSION

The project, *Ramanujan's Magic Square*, combines mathematical learning with interactive puzzle-solving in a web-based application. It introduces users to magic-square rules, the magic constant, construction methods, and Ramanujan's date-based 4×4 square. Through Swap, Fill, and Manual modes, users can practise arranging numbers and checking whether the required sums are correct.

The application also supports continued learning with hints, undo, Daily Challenges, achievements, progress tracking, leaderboards, and a chatbot for MagicSquare and website questions. User accounts and SQLite storage allow profile and gameplay information to be retained, provided the deployment uses persistent storage. AI chatbot responses are available when the required API configuration is enabled; otherwise, the chatbot uses curated responses.

Overall, the project demonstrates how mathematical concepts can be presented through an accessible, interactive website. It provides a foundation that can be extended with further usability improvements, broader learning content, and additional testing. Any claims about speed, reliability, or user performance should be supported by measured results from actual testing.

CHAPTER 15: DASHBOARD SCREENSHOTS

15.1 Home Page and Account Entry

[SCREENSHOT PLACEHOLDER — Home page and account entry: image not supplied]

Figure 15.1: Home page and account entry screen.

15.2 User Registration and Login

[SCREENSHOT PLACEHOLDER — Registration and login: image not supplied]

Figure 15.2: User registration and login screen.

15.3 Player Dashboard

[SCREENSHOT PLACEHOLDER — Player dashboard: image not supplied]

Figure 15.3: Player dashboard showing progress and achievements.

15.4 Profile Dashboard

[SCREENSHOT PLACEHOLDER — Profile dashboard: image not supplied]

Figure 15.4: Profile dashboard and account-management actions.

15.5 Magic Square Gameplay

[SCREENSHOT PLACEHOLDER — Gameplay interface: image not supplied]

Figure 15.5: Magic-square gameplay interface.

15.6 Daily Challenge and Leaderboard

[SCREENSHOT PLACEHOLDER — Daily Challenge and leaderboard: image not supplied]

Figure 15.6: Daily Challenge and leaderboard screens.

15.7 Formula and Learning Section

[SCREENSHOT PLACEHOLDER — Formula and learning section: image not supplied]

Figure 15.7: Formula and learning section.

15.8 Admin Dashboard

[SCREENSHOT PLACEHOLDER — Admin dashboard: image not supplied]

Figure 15.8: Administrator dashboard and review interface.

REFERENCES

[1] G. H. Hardy, Ramanujan: Twelve Lectures on Subjects Suggested by His Life and Work, 3rd ed. New York: Chelsea Publishing, 1978 (originally published 1940).

[2] R. Kanigel, The Man Who Knew Infinity: A Life of the Genius Ramanujan. New York: Charles Scribner's Sons, 1991.

[3] B. C. Berndt and R. A. Rankin, Ramanujan: Letters and Commentary. Providence, RI: American Mathematical Society, 1995.

[4] B. C. Berndt, Ramanujan's Notebooks, Part I. New York: Springer-Verlag, 1985.

[5] C. K. Ghosh, "Fun With Magic Squares," Dream 2047, vol. 18, no. 8, May 2016, pp. 32–33, Department of Science and Technology, Government of India.

[6] Community Mathematics Centre, "A Magic Square in Ramanujan's Honour," At Right Angles, vol. 4, no. 3, Nov. 2015. Available: https://publications.azimpremjiuniversity.edu.in/3104/

[7] W. S. Andrews, Magic Squares and Cubes. Chicago and London: Open Court Publishing Company, 1917.

[8] J. R. Hendricks, Magic Squares to Tesseracts by Computer. Self-published, 1998.

[9] D. M. Brown, "The Magic Squares of Manuel Moschopoulos," Convergence, Mathematical Association of America, 2012. Available: https://old.maa.org/press/periodicals/convergence/the-magic-squares-of-manuel-moschopoulos-magic-squares-before-moschopoulos

[10] A. Eves, "The Fascination of Magic Squares," Royal Institution, Primary Mathematics Masterclasses, online article. Available: https://www.rigb.org/explore-science/explore/blog/fascination-magic-squares

[11] M. R. McKay and C. J. Radcliffe, "Enumeration of 4 × 4 Magic Squares," arXiv:0907.3188, 2009. Available: https://arxiv.org/abs/0907.3188

[12] K. Ono, "Ramanujan, Taxicabs, Birthdates, Zipcodes, and Twists," expository lecture notes, University of Virginia. Available: https://uva.theopenscholar.com/files/ken-ono/files/023_8.pdf

[13] "Srinivasa Ramanujan," MacTutor History of Mathematics Archive, University of St Andrews. Available: https://mathshistory.st-andrews.ac.uk/Biographies/Ramanujan/

[14] "Hardy-Ramanujan Number," Wolfram MathWorld. Available: https://mathworld.wolfram.com/Hardy-RamanujanNumber.html

[15] User-provided document: WhatsApp Image 2026-10-24 at 10.50.23 PM.pdf, academic project-report model, 53 scanned pages. Consulted for report structure and register only; unrelated subject content not used.

[16] User-provided image: maxresdefault.jpg, handmade 4 × 4 date-square teaching display, supplied with the project brief. Reproduced in Figure 9 for explanatory context.

[17] User-provided images: OIP.webp, OIP (1).webp and sddefault.jpg, supplied with the project brief. Reviewed for the square’s teaching presentation; not reproduced because the report redraws the groups and entry grid for legibility.

Access dates for online references should be added if required by the department’s citation policy. Book editions and publication details should be checked against the copies consulted by the student before final submission.

BIBLIOGRAPHY

Use a numbered reference style in the report and cite these sources where relevant in the chapters.

[1] MacTutor History of Mathematics Archive, “Srinivasa Aiyangar Ramanujan,” University of St Andrews. Available online. Accessed 30 October 2026.

[2] S. B. Block and S. A. Tavares, “History of Magic Squares,” in *Before Sudoku: The World of Magic Squares*, Oxford University Press, 2009, pp. 20–32. Available online.

[3] “Magic Squares and Other Problems Upon a Chess-Board,” *Nature*, vol. 65, pp. 447–452, 13 March 1902. doi:10.1038/065447a0.

[4] “A Magic Square in Ramanujan’s Honour,” *AtRiA: A Magazine on Mathematics*, Azim Premji University, vol. 4, no. 3, November 2015. Available online.

[5] Department of Science and Technology, Government of India, “Dream 2047,” May 2016, vol. 18, no. 8. Available online.

[6] J. Sesiano, *Magic Squares: Their History and Construction from Ancient Times to AD 1600*. Springer, 2019.

[7] Pallets Projects, “Define and Access the Database,” *Flask Documentation*. Available online. Accessed 30 October 2026.

[8] Python Software Foundation, “sqlite3 — DB-API 2.0 Interface for SQLite Databases,” *Python Documentation*. Available online. Accessed 30 October 2026.

[9] SQLite Consortium, “SQLite Documentation.” Available online. Accessed 30 October 2026.

[10] *Ramanujan’s Magic Square Project*, source code and project documentation: app.py, chatbot_rules.py, chatbot_i18n.py, README.md, and knowledge/magic-square-guide.md

Appendix

Appendix A: Ramanujan's Date-Based 4×4 Magic Square

The following square uses the digits of Ramanujan's birth date, 22 December 1887, in its first row. Each row, column, and main diagonal sums to 139.

Table 11: Ramanujan date-square reference grid

22 12 18 87

88 17 9 25

10 24 89 16

19 86 23 11

For example, the first row is $22 + 12 + 18 + 87 = 139$, and the main diagonal is $22 + 17 + 89 + 11 = 139$. The four corners and the central 2×2 block also sum to 139. The number 1729 is associated with a separate well-known story about Ramanujan; it is not an entry in this square.

Appendix B: Magic Constant Values

For a normal magic square of order n, the magic constant is:

$$M = \frac{n(n^2 + 1)}{2}$$

Table 12: Magic constants for supported normal-square orders

Order Magic constant

3×3 15

4×4 34

5×5 65

6×6 111

7×7 175

8×8 260

9×9 369

10×10 505

Appendix C: Square Validation Pseudocode

```
function validate(square, order):
  target = order * (order * order + 1) / 2
```

```
if square does not contain order × order cells:
    return invalid

if square does not contain each integer from 1 to order² exactly once:
    return invalid

if any row sum differs from target:
    return invalid

if any column sum differs from target:
    return invalid

if either main diagonal sum differs from target:
    return invalid

return valid
```

Appendix D: Project Configuration

The application uses environment variables for deployment-specific configuration. Values such as secret keys should be set privately on the server and not committed to the source repository.

Table 13: Application environment variables

Variable	Purpose
SESSION_SECRET	Protects application sessions; set a unique, secure value for deployment.
DATA_DIR	Specifies the directory used for persistent SQLite data.
OPENAI_API_KEY	Enables AI-powered chatbot responses when configured.
OPENAI_MODEL	Selects the OpenAI model; the project documentation lists gpt-4.1-mini as the default.
ADMIN_USERNAME	Identifies the username configured for administrator access.

Screenshot materials to supply

The source document did not include dashboard screenshots. Replace the marked placeholders in Chapter 15 with the corresponding website images:

- Home page and account entry
- User registration and login
- Player dashboard
- Profile dashboard
- Magic-square gameplay
- Daily Challenge and leaderboard
- Formula and learning section
- Admin dashboard

In Microsoft Word, update the Table of Contents, List of Figures, and List of Tables fields after inserting screenshots (Ctrl+A, then F9) so page references reflect the final layout.